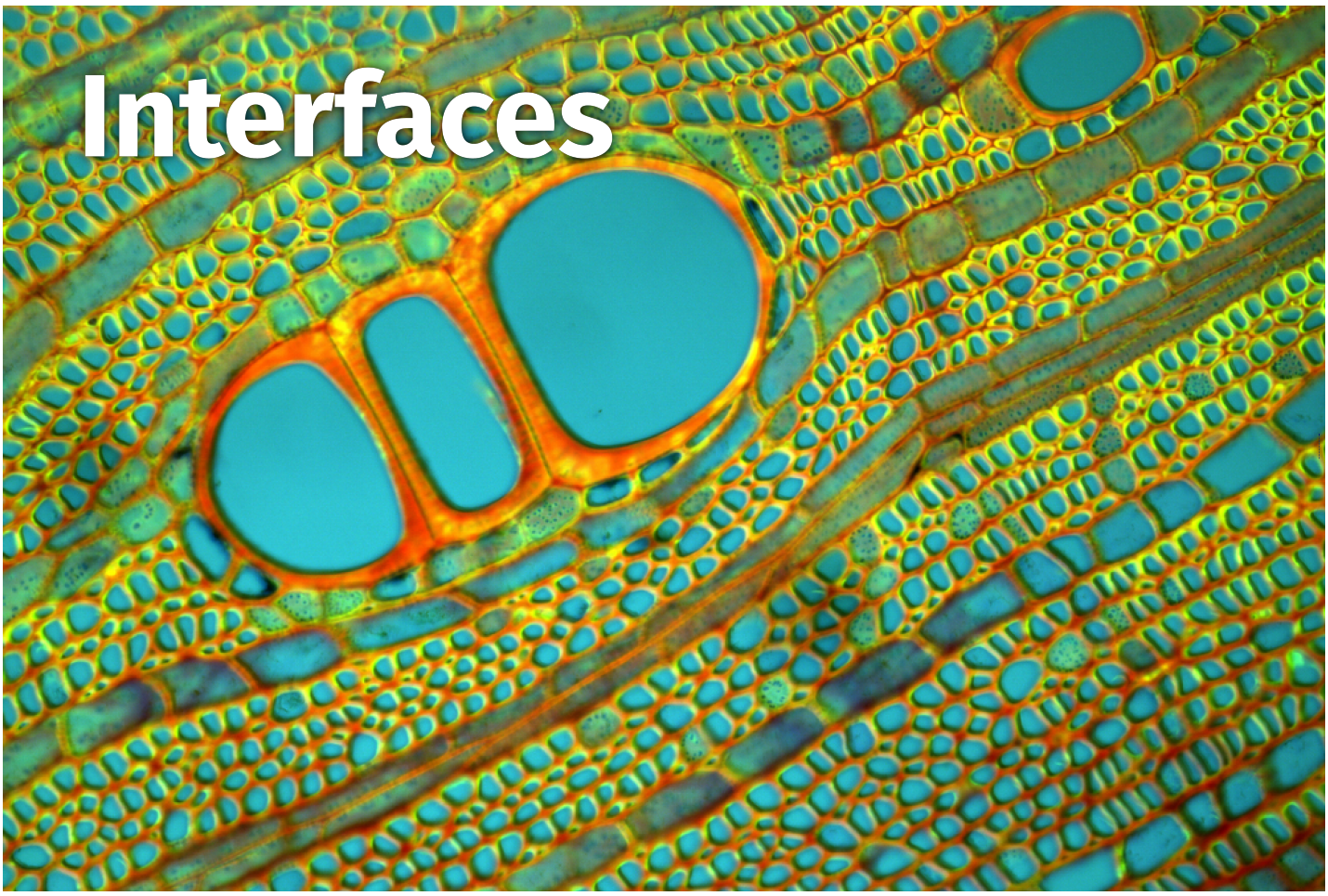
Software Architecture

Reusability and Interfaces

6

Contents

- Required and Provided Interfaces
- Information Hiding and Encapsulation
- Interface Description Languages
- Tradeoffs: Usability vs. Reusability vs. Performance
- APIs: Examples and Design Advice
- API Design Principles: Explicit Interfaces, Least Surprise, API First, Small Interfaces, Uniform Access, Few Interfaces, Clear Interfaces



Interfaces

After we have seen components and how they make the architecture of our system modular, today we are going to discuss more in detail, how do the interfaces of components look like and how they make it possible to build the architecture out of reusable elements.

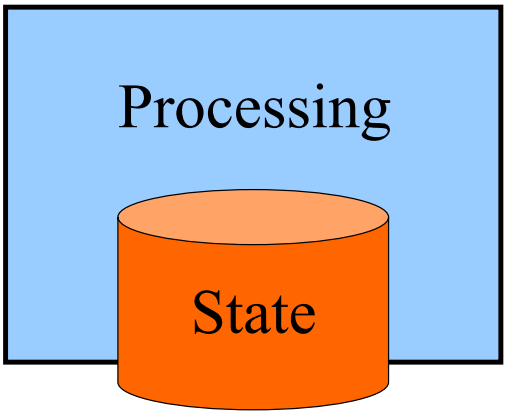
The main element that makes it possible to reuse a component is its interface.

I would like to begin with this picture showing you a different type of interface, not the interface or piece of software, but the interface a biological system.

How is it possible to compose a larger organism out of individual cells? Cells are separate entities, but they can live together in multi-cellular organisms, thanks to the interface, which separates them but also allows them to communicate and exchange stuff. The interface or the cellular membrane also separates living space inside from the outside world. It acts as an extremely thin filter, which for example, is also very important to keep viruses out.

Component

- Locus of computation and state in a system

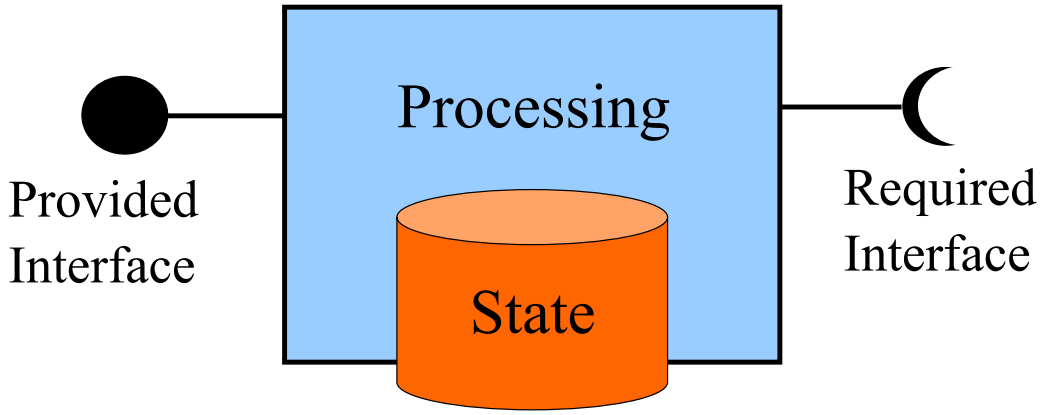


- Reusable unit of composition

The last lecture was about modularity and components. Components are the result of the structural decomposition of your architecture. They are responsible for processing and storing data. Thanks to their interfaces, components are also meant to be composed as a building block and assembled into larger software architectures.

Component Interface

- How to use a component?



- How to reuse a component?

If we focus on the interfaces, the main concern becomes: How do we use the components which make up our architecture? How do we make their data processing logic and their state accessible to other components so that developers can build and integrate them into larger systems?

We use components through their interface which gives us access to the functionality they provide.

To make it possible to reuse a component, we also need to satisfy its dependencies. A component that is missing some dependencies will not work. It may refuse to start. Or if for some reason it did start, you try to call it and the component will crash.

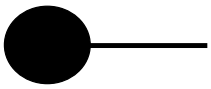
How to use the component? How to reuse the component? These are not the same.

From the perspective of a component vendor, the goal is not only to make it usable for one developer within their specific context, but to make it reusable so that a lot of customers want to buy this component.

If you want to make a component usable, you need to invest into making the provided interface as simple and as easy to learn as possible. A general and widely applicable interface will instead make your component more reusable.

If you want to use a component then you also need to focus on making sure that its dependencies are satisfied. If you want to make a component reusable, then it should not have any dependencies. Your customers can take your component and start using it immediately without bothering about installing any dependencies.

Both use and reuse are interdependent. Start by making the component usable in one system, for one customer, for one solution. And then try to generalize its interface and make it applicable to multiple domains. Turn it into a generic, highly reusable, infrastructure component which everybody cannot do without for building any kind of system.

Provided Interface 

- Specify and document how to use the externally visible features offered by the component:

1. Data Types
    - Define the component data model
2. Operations
    - Call functionality, modify state
3. Properties
    - Read/Write visible state attributes
4. Events
    - "Call-backs" – Notify about state changes

Let's look at each side starting from the provided interface. This is the interface through which we get access to the functionality of the component.

This is the surface on which we need to make internal features visible so that developers can use them. To do that, you need to have some kind of a specification, some documentation that explains how to use this component.

What kind of information do we need to include in the provided interface? We're going to define the data model for the component: what kind of data types are going to be used to represent the information that is exchanged as input or output via this interface.

This is typically done using the type system of some programming language, using it to define a data model. It's also possible to use some programming language agnostic type system, for example XML or JSON or Unicode character strings.

The goal of the interface data model design is to define what exactly is the representation of the information that I can exchange with the component. Depending on the data formats that you select, you can be more or less compatible, more or less interoperable.

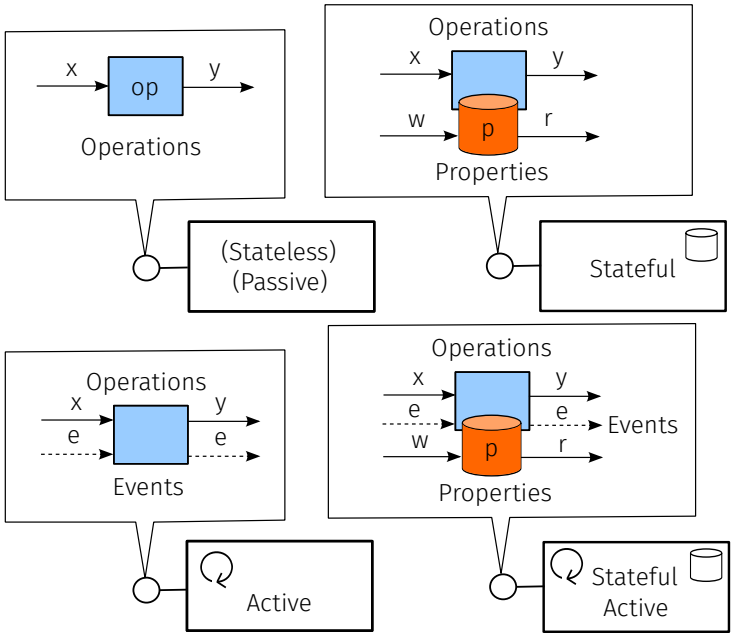
The second aspect comes into play for components which are not just repositories for storing information, they are also active elements that can process and transform the data. So the interface should enumerate which operation the component offers. How to call the functionality implemented by the component? Is the call going to have an effect on the state of the component? or is this a purely functional operation?

For stateful components, you also want to access the state of the component and this is what we do through the interface. The content of the storage within the component can be directly accessible: you will be able to read and write properties, or state attributes that have a certain type according to the data model.

The interfaces of active components will also feature events. They notify the outside of the component that something interesting has just happened inside. The goal of events is to make it possible to observe through the interface internal state changes of the component.

Data Model, Operations, Properties, and Events: these are the four basic features that you may find within the provided interface of a component.

Provided Interfaces and Component Roles



Which type of components provides each of these features? We can classify components as being stateless or stateful as well as active or passive. These are orthogonal, so we can have: passive and stateless, passive and stateful active and stateless, and active and stateful.

A stateful component remembers the entire history of its inputs. The output of interaction with the stateful component depends not only on the input that you provide, but also on the previous inputs.

Active component start running a thread of control, and therefore it can do things on its own. Passive component doesn't do anything unless another component calls it.

Starting with the simplest type of component, the one that is stateless and passive, its interface will only feature basic operations which take some input, process it and give you back the output. This will happen only when you call the operation from the outside. Since there is no state, the component will not remember that you did that, so every time you call it with the same input again, you can expect the result will be the same. The passive component will not execute the operation unless you call it from the outside.

In case the component is stateful, then we have to also consider the fact that inside the component the effect of the operations will be stored. Not all stateful components have properties, since their internal state can be indirectly manipulated via operations.

But it is also possible to access directly the internal state. If you decide to expose the internal state of the component at the interface level, that's what properties are for. When you set the value of a property you are going to modify the corresponding internal state. When you read the property you will have the chance to directly check what is the current state of the components. That's what properties are for: to offer direct access to the state.

Sometimes properties are not present in the interface even if the component is stateful, because when you call the operation, the operation can modify the state but also give you access to indirectly read its current value.

If you don't have a stateful component, then properties at the interface level are useless because even if you set the value of a property, the component doesn't know where to store the value.

If we look at the case in which we have an active component with some background thread running inside, then it becomes interesting to make the effects of such thread visible at the interface level through events.

When you call an operation, you typically give the input and then you wait for the output. And the thread that calls the operation jumps into the component, executes the code and then returns control back to you.

If you have a thread inside the component, then you can just work with events where, for example, you submit the input into a queue. And then you can continue doing something else when the internal thread of the component becomes available, takes the input from the queue, processes it, and as it gives back the output, it notifies you with an event that there is a result and then you can pick it up at your earliest convenience.

Events make sense to be present in the interface if you need some asynchronous interaction with an active thread of control inside the component, behind the interface. You can find events in the interface of active but stateless components: whenever you submit some input, it will take it and start a thread to process it in the background and give you back a notification when it is done. There is no event queue, since the component is stateless. Also there is no memory about the past, so every time it will start processing the input from scratch, without the possibility to cache previous results.

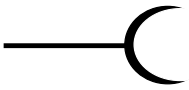
If we combine these two, we have a stateful and active component. This is the most complicated one, with the richest possible interface, featuring: operation (blocking), events (asynchronous interactions), as well as properties (direct access to the internal state).

I hope that this example has explained even more the difference between operations, events, and properties.

For all three, you will need to also define what is the corresponding data model. If you have an event: what's the data associated with it? is it a pure signal (just an identifier to distinguish it from other events) or does it carry a message (with a complex payload)?

For properties even more so, how do you expose a projection over the internal state? what kind of data structure do you see from the outside through the interface?

It is your decision as an interface designer to use these elements. You can also have a stateful active component which only features operations in its interface. It will not be possible to observe or alter directly its internal state, since properties are missing. All interactions through operations will block the caller, since event notifications are missing.



Required Interface

- A component can be reused only if its dependencies are satisfied
- The platform is compatible:
  - Runtime Libraries/Framework
  - Operating System/Device Drivers
  - Hardware
- The environment is setup correctly:
  - Databases
  - Configuration Files
  - File System Directories

The required interface describes what a component needs to work properly. It answers the question: what are the dependencies which we should satisfy?

We find the information that tells us what the component needs in its required interface.

The required interface can be satisfied by introducing another component that provides exactly the interface required by our component. If we can connect the two interfaces together we have successfully satisfied the required interface. This needs to be solved recursively, so now let's look at how to satisfy the required interface of the newly introduced component.

The required interface can have broader requirements. A component may only work in a certain runtime environment. They need to be deployed on a certain platform. They require a certain version of the operating system. They might need some particular device drivers. Maybe the components are directly dependent on the presence of some particular hardware, some sensor, some type of processor, or GPU.

These are also all dependencies, which may get overlooked. Nowadays we are used to write portable software components, which do not make assumptions about which exact version of the operating system they run on. Sometimes they don't even care about which exact operating system they will run on, as long as there is one.

These environmental dependencies can be significant deal breakers. You have a component that you want to use, and then you discover that it doesn't work without a certain device driver of the operating system. Either you upgrade (or downgrade) the operating system or you find another component.

You really wanted to install the official national covid tracking app, but it turns out it needs access to the patched bluetooth API which has been only included in a recent release of your mobile OS, which unfortunately doesn't seem to run on your device.

Sometimes these hard to satisfy dependencies do not come from the component itself.

The component could perfectly use an old version of the operating system as it is not using any of the recently added APIs. It turns out that the compiler that you use to compile the code of your component only supports a certain version of the operating system as target to run the code that it produces.

You will be surprised how subtle can dependencies be and how mostly harmless architectural decisions (such as the choice of your favourite programming language) will limit where your software can run.

Another source of unexpected dependencies come from the dependencies of your dependencies. Those are also difficult to control. You can control the immediate dependencies from a component, but if this component needs other components, they also have their own dependencies and your system will work only if the transitive closure of all the dependencies is satisfied.

More dependencies: Not only we have to pick the right platform, but we also need to set up the system with the right configuration. For example, if you have a stateful component, it will need to connect to a database to manage the storage of its state. The component needs to know: where is the database? How do I connect to it? What kind of database account am I going to use? And if you if you start the component without this information, as soon as the component will attempt to store some data, it will fail to locate an appropriate database and will crash or continue to work in a zombie-like state with the unsatisfied assumption that its state has been stored persistently.

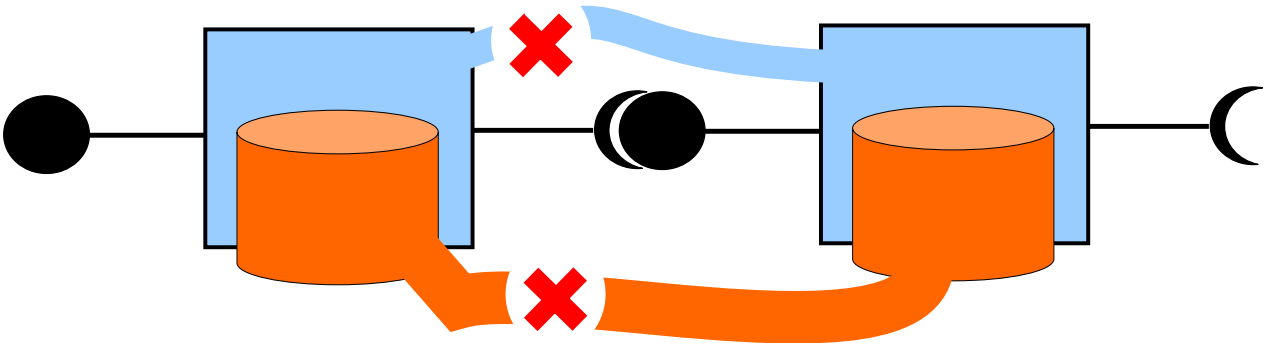
Such configuration needs to be written in some configuration files. Storing the configuration in the database itself opens up a recursive meta-configuration problem, which is better to avoid. A very frequent reason for initialization crashes or startup failures of components is that the configuration files are missing, incomplete, corrupt or incorrect. You hope that if you leave some configuration default and but also the default configuration is not there.

Failure to satisfy required interfaces impacts the ability to build but also deploy and start operating your component. You need to make sure that you deploy it in a compatible platform and you also configure its runtime environment correctly.

As an architect first you try to maximize the component compatibility with as many platforms as possible. Try to avoid restricting it unnecessarily. As a developer, for example, you should write your code in a defensive way, where you always give defaults to configuration options and you do not crash if the directory that you expect is not present. Instead, you can just create it on the fly so that everything works smoothly and you can switch on the system and start it the very first time without any problems.

Dependencies: we tend to forget that these exist, but the moment that you take a component that you didn't develop yourself and you want to buy it, for example, knowing detailed information about its required interface is critical because otherwise you risk that the software you just bought will never work for you. It did work for the original developer though.

Explicit Interfaces Principle



Bertrand Meyer

- Components always communicate through interfaces
- No Shared State
- No Out-Of-Band Communication

When we talk about interfaces, we make a fundamental assumption: in our design and in our implementation the only communication channel between components is the interface.

It is tempting when you write the code to ignore that assumption. What if you want to call an operation hidden inside a component and not visible in its interface. Because you are using a certain language that doesn't properly encapsulate private functions, you can still – if you know where to look – find the operation and make the call.

For some reason, they forgot to make it accessible through the interface, but your quick solution solved the problem: you got to what you need.

Another temptation is to just store the partial result in a global variable so that all your components can now happily share the information they need without having to expose it through properties or getter operations in their interfaces.

Why do you think it's a bad idea to do this? What could be a consequence when you introduce a global variable? You're breaking the encapsulation that the component boundary gives you. You break the independence of the components you use. Their interface is no longer truthfully stating which are the available operations, which are the points in which the component can read input or write its output results.

Now the components sharing a global variable are strongly coupled because of the shared global variable. If one component changes it, every other component (it's a global variable after all) will see the effect. Is this what you really want? If you access data protected by an interface, you could assume that if there is concurrent access, the interface takes care of the serialization to keep the state of the component consistent. But if you just bypass that what do you expect will happen in case of concurrent access to a global variable shared by multiple threads?

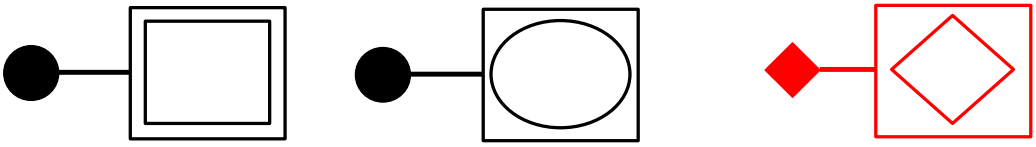
At the end, if you violate the explicit interface principle, you no longer have separate components. If you bypass their interface, you introduce direct coupling between the outside and the inside. If someone manages to patch through a call to a private function, this function is no longer private. You are no longer free to evolve it independently of the rest of the architecture. Everyone could be calling this function. How do you know?

If learning how to use a component through its official interface is hard enough, how hard do you think it is to learn how to use it by going through its entire implementation? What if someone else implemented the component?

The interface concept results from lessons learned in many decades of software architecture. We are still arguing on how to design them properly, but we all agree they are important, powerful and useful. Bypass interfaces at your own risk, you have been warned. Your architecture will never go beyond a big ball of mud or spaghetti monster, where everything is connected to everything else.

Information Hiding

- Always keep the implementation details secret (invisible from the outside) and only access them through the public interface



- Makes it possible to replace components (as long as they share the same interface)
- Ensures component decoupling

The second, very important, aspect of interfaces is about their purpose to abstract the internal implementation of components. You just work with the interface, but you don't care about what happens inside the coffee machine. As long as you put in the money, select the amount of sugar, then you get the result of the computation. No need to worry about where is the source of the electrons boiling the water, or which part of the world the coffee was harvested from.

The goal of establishing an interface is to raise the abstraction level so much that for example you can no longer tell if the task you submit is implemented by a machine or by a human intelligence. Was this lecture recording transcribed by a human listener or automatically generated? Does it matter if you can just read it in this book? This was at some point called the Mechanical Turk, which literally worked thanks to a person hidden within the box, moving the automaton.

If you do not have an interface then everything will be visible, you will need to consider how is the functionality implemented if you want to learn how to use it.

Thanks to the interface, we can ignore the implementation details and only focus on how do we use the component without having to worry about how the component implements what we're going to use. We use the term "black box", as a metaphor for a box whose inside is not visible. We are actually not interested in looking inside the box.

What is information hiding about? The goal is that since you have an interface, you keep the implementation secret and hide it from the outside. And why this is important? As long as the interface doesn't change, you can replace the implementation. You can switch the implementation as long as the interface is not affected.

You establish a dependency to the interface in order to avoid establishing a dependency to the implementation.

Information hiding ensures that who is using your components remains decoupled from your implementation. They are, of course, coupled to the interface, but the point of the interface is to separate them from what is behind the interface, which is the implementation, which remains a secret thanks to the interface.

This separation works both ways. So who is using your component will not depend on the implementation. But your implementation should also not depend on who's using your components. The only thing that is in common is the interface, and both are dependent on it.

If we change the implementation so much, it is no longer possible to contain the effect and the change will leak at the interface level. If this happens, every other component depending on the interface will be affected.

Effective Encapsulation

- Reduce the cognitive burden for using a component
- Well designed interfaces are simple to understand
- Every interface should hide one secret (avoid leakage: do not reveal unnecessary details)
- Changes to the implementation hidden behind an interface do not affect clients
- Difficult to hide the impact of all changes
- How to choose what should be hidden?

When you encapsulate a component and abstract it through the interface, you make it easier to use the component because you just have to understand its interface and you can ignore the implementation details.

We will discuss what are the qualities of well designed, easy to understand interfaces.

The core of what you do as a interface designer (one of the various tasks an architect should work on) is to avoid leaking implementation details at the interface level. You don't want to reveal details that are unnecessary if you want to use the component. Make life easier for the developers using your component, it was already hard enough to successfully implement it. You don't have to worry about all of these things, look, I just hide them from you.

The advantage is that – again – if you change the implementation and the implementation change did not affect the interface, then you know you have managed to contain the impact of your change within the boundaries of the component.

In general, it is impossible to hide the impact of any arbitrary change that you make to an implementation. What if your change requires to produce or consume more data. What if you are now changing the nature or the semantic of an existing operation and make it compute something different, something new, something unexpected?

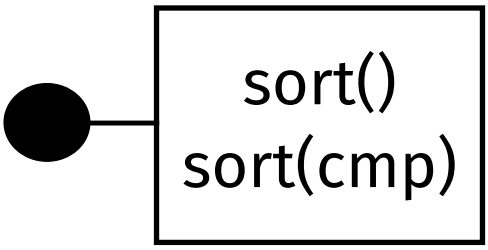
I modified the implementation of an existing operation, but now I need more parameter. The operation is still the same, but callers have to send along one more parameter. How can you mitigate this interface change? What if you can give a default value for the new parameter? If they don't give it to me, it still works thanks to the default. And if they want to take advantage of the new version, which is more powerful, they need to change their code and they have to pass the new parameter.

And your task as an interface designer is to make these decisions. Do you start with an existing component, which is already implemented and then try to abstract its interface? What are you going to hide? What are you going to reveal?

This are difficult design decisions to make, because they mainly depend on who is on the other side: who wants to use your component. They will need certain things and will expect to find them in the interface. Nothing else.

Example

- Pre-condition: unsorted content
- Post-conditon: sorted content
- Hidden details:
  - What sorting algorithm is used?
  - Is the sorting algorithm stable?
  - What if the content is already sorted?
  - What data structure stores the content?
  - Can I provide my own comparator strategy?



In this example we have a very simple interface with an operation which transforms an array whose elements are in some random order into a sorted array.

That's all the information that you have to describe what's in the interface.

What do you think this abstraction is hiding about the implementation?

We are hiding: the sorting algorithm. The complexity of the sorting algorithm. From the outside, when we call the sort operation, we don't really know about these details. What's the performance we observe, can we send different array sizes and try to reconstruct the algorithm based on the time it takes to process the input?

It is also not specified the sort order: ascending or descending?

What about: how the values are stored? Are we working with an array with a linked list, or with a double linked list? Is there a limit on the size of the input array? This would belong into the data model of the interface. Maybe the implementation code is independent from how the input data is structured. But definitely writing the sort algorithm code for an array is not the same as implementing it for a list.

Let's change the interface: now it exposes one more parameter. This is a comparator function, which helps to sort arrays where the elements store arbitrary content. As long as you know how to compare a pair of values, you can sort the whole array.

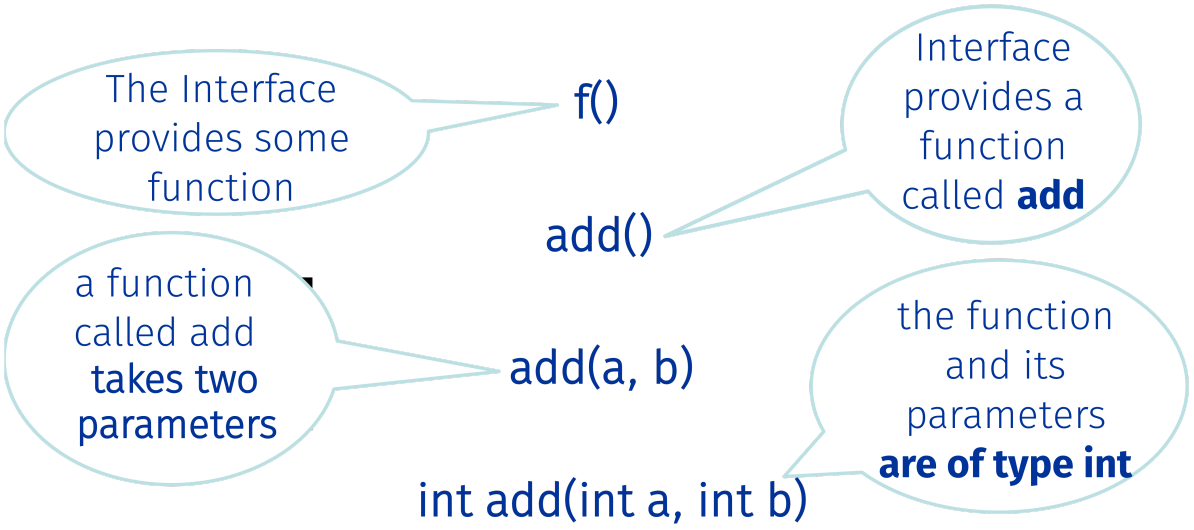
This is another assumption: does the implementation know how to compare and evaluate the order of two arbitrary elements. Making this part of the interface, helps to shift this knowledge originally buried inside and make it accessible to the outside.

Another assumption: What if the input array is already sorted? Can this be detected? If so, can the actual sort be skipped?

One last assumption concerns the stability of the algorithm. Do equivalent or identical elements retain the same position? or will they be shuffled by the sort even if they have the same order?

What is also not fully clear is whether this is the interface of a stateful component, where the array to be sorted is stored within the component itself, or it is an operation of a stateless component, which would be applied to an array given as input and forget about the result afterwards.

Describing Interfaces



---

add(a, b) {return a * b;}

After abstracting away the implementation to satisfy the needs of the developers trying to reuse our component, what is left?

Whatever is left, we need to describe it.

Let me give you an example to show to which extent or how precisely you can give such a description and why it's important to do so.

If we just list the operation but we do not give its name, we know that it's possible to invoke the operation 'f()', but we don't know exactly what the operation does, we just know it exists.

The lack of name is ambiguous and imprecise. If we give it actually a name, through the name, we already share a lot of information and create the expectation that this operation will compute something: it will perform an addition. This is a step forward, but it's still not enough. Do we really know how to use it? Do you know exactly what is going to add? Is it a stateful component which will do an addition over some values already stored inside the component? But if the component is stateless, then we need to know more: We need to know how do we give the input data so that we can process it and perform the computation.

So now we know that actually this operation is a function that can add two parameters. Is this enough or do we need to further refine our description?

Out of all the possible pairs of parameters that we can pass, we should specify also their type. So this refers to describing the data model: is it going to be an addition over floating point or integer numbers? What if you pass strings? will it try to concatenate them?

It depends, we have data types for the input parameters, but we should also specify them for the output result.

Here we have now reached the full operation signature with the types and names. If you are going to implement the component with a statically typed programming language, you have now enough information to write the code.

The previous examples were enough to use the component and call it from a more loosely typed programming language.

Having a detailed specification of a statically typed interface helps to check statically that the interface that you have selected is compatible with your code.

The description makes it possible to verify the corresponding expectations at compile time or at run time, just before making the call: the function with the given name exists, the parameter names (and types) match. The less assumptions are explicitly described, the more difficult is to do any validation and the incompatibilities will be discovered only at runtime.

The ultimate truth about what this interface does lies actually in the implementation itself: the only place where you truly know what is the actual relationship between input and the output.

In this example, do you see something unexpected? There is a little bit of a surprise. There is a small inconsistency between the multiplication expression that we have in the implementation and the name of the function that it says that we're going to do an addition.

Principle of Least Surprise

> Interfaces should behave consistently with the client's expectations

- Obvious
- Consistent
- Easy to guess
- Natural
- Predictable
- Easy to learn

The name of a function should reflect what it does

The name of a parameter should indicate what it means

The principle of least surprise should be respected when abstracting and describing interfaces.

Especially when you pick the names that you use inside the interface, make sure that they create the right expectations in the developers reading your description.

This expectation needs to match what you find in the implementation. If you call the function called ”addition“ but you multiply the numbers, you violate this principle.

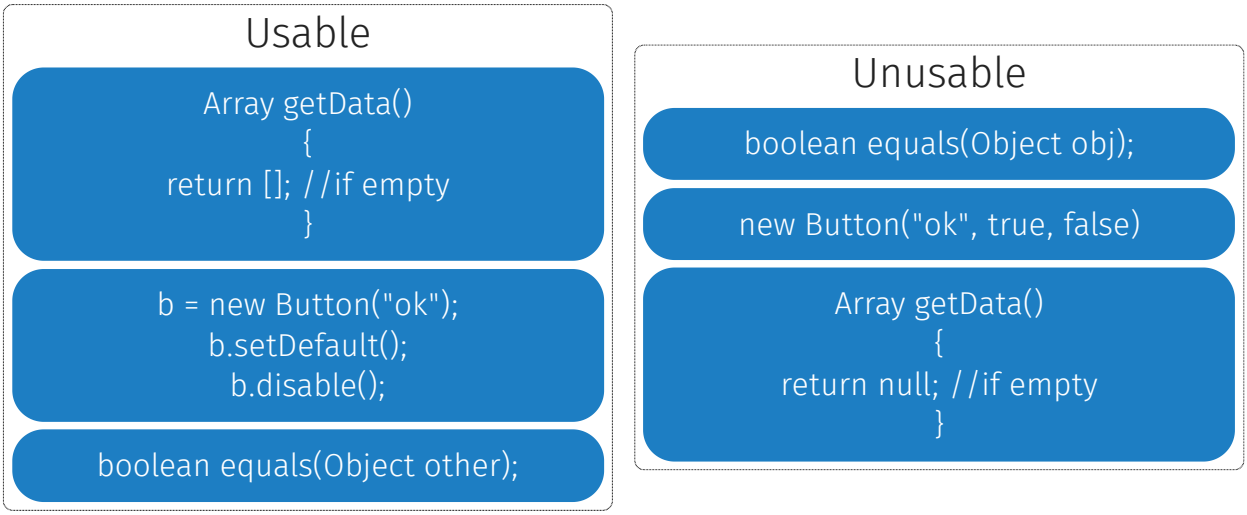
The principle of least surprise is important not only for consistency, but also for making things predictable, easy to learn, easy to guess.

If you give a name, the name should help to explain what the corresponding interface feature does. Giving names to interface elements is most critical: names (and the names of types) should compress the intended semantics in one CamelCase word.

You can use additional documentation, such as natural language descriptions to mitigate the effect of incorrect names, clarify ambiguous ones, and shed more light on the semantics and usage scenarios of your interface elements. Still, even without such additional metadata, the names you pick should speak for themselves.

Not to add any more pressure, but: once you come up with a name, give it to some interface element, and release the interface, it will be difficult to change the name once others start using in. A wrong or ugly name will stain your interface for a long time.

Easy to use?



About making the interface easy to learn and easy to use, I wanted you to reflect about these various examples: find a pair that is more usable than the other one. Some are specifications of interfaces, others are examples of usage for the interface, others also include the implementation.

We can start from the new button interface. There are three parameters: the first could be easy to guess: the text that you put inside the button. What about the others: true, false. How can you tell what they mean?

Now if we look at the alternative, we're using properties to configure the button after we construct it with its text: it's the default button and it's disabled.

So by transferring what we know from here, we can guess that the 2nd parameter indicates whether the button is disabled or not; we disable it because we set it to false.

If you just read the code, reading these boolean flags, especially if you find a lot of them, makes it hard to understand what the code means unless you remember the position of each flag. That's not a nice practice when you design your interface, unless you rely on people having a very good memory about the order of the bits they should pass to configure a button.

In the alternative the interface is more user friendly because we use the names to give meaning to those boolean flags.

The second example is about minor difference on how to name one parameter: obj vs. other. This can impact the interpretation of the same "equal" method: is it equal to other, or is it equal to object.

Sometimes we give names to parameters that are just a placeholder for their type. The language gives us two independent elements, the name and the type, but we only use the type: you already know it is an object from its type the name 'obj' does not convey additional information. If we name it 'other', we give more meaning: the comparison equal checks the parameter against the state of the current object. So that is slightly more clear.

The third example is a piece of code that implements the 'getData' which returns an array. Think about how you would write the caller of this code. When you retrieve an array you typically want to iterate over it. How do you deal with the empty array case?

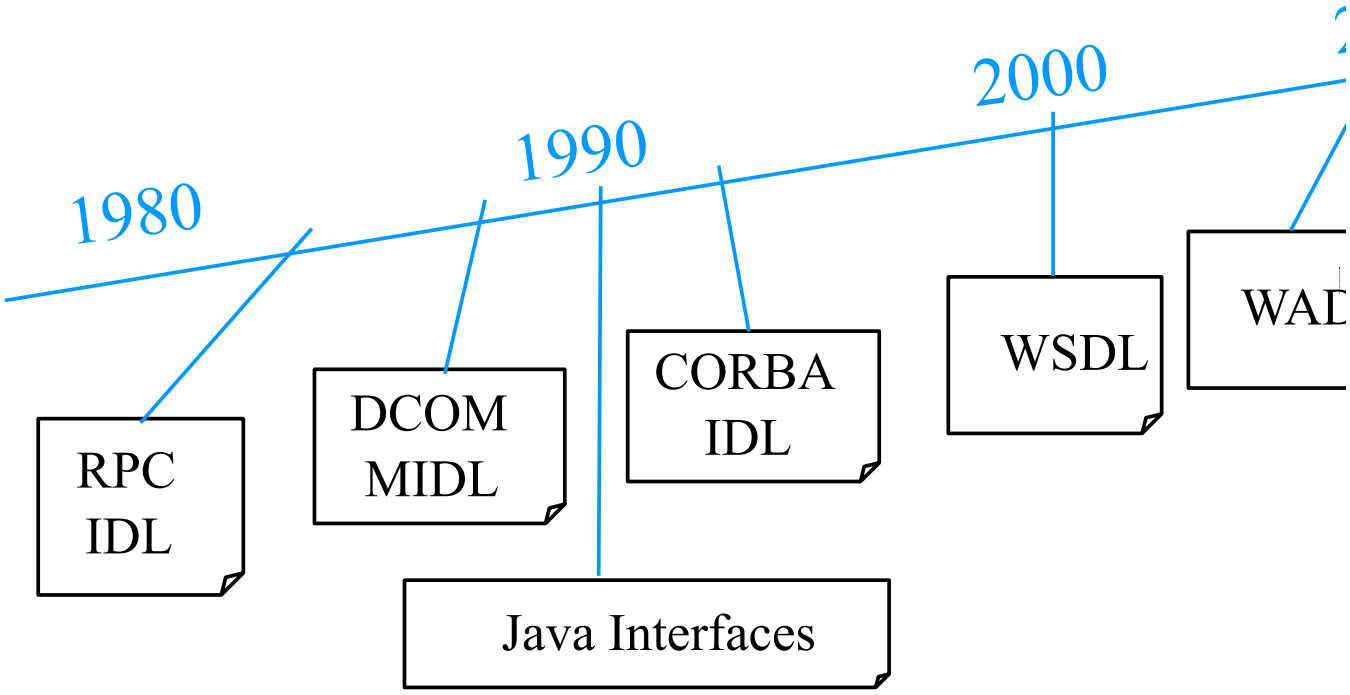
Is it easier to write a client which can just iterate over an empty array, i.e., do not process any element if none are there? or do you prefer to write code which tests for a special case, the null result, and thus explicitly skip executing the loop if the data is empty?

While there is a different between the semantics of missing data (null) from empty data ([]), you should be aware of this difference and do not use one for the other. While it is better to return empty arrays so clients can process them as array without having to detect and deal with null values, in general every null or nil value that an interface can return generates a potential crash in the client code, which somewhere (not necessarily right after getting the null result) may forget to protect against dereferencing null values.

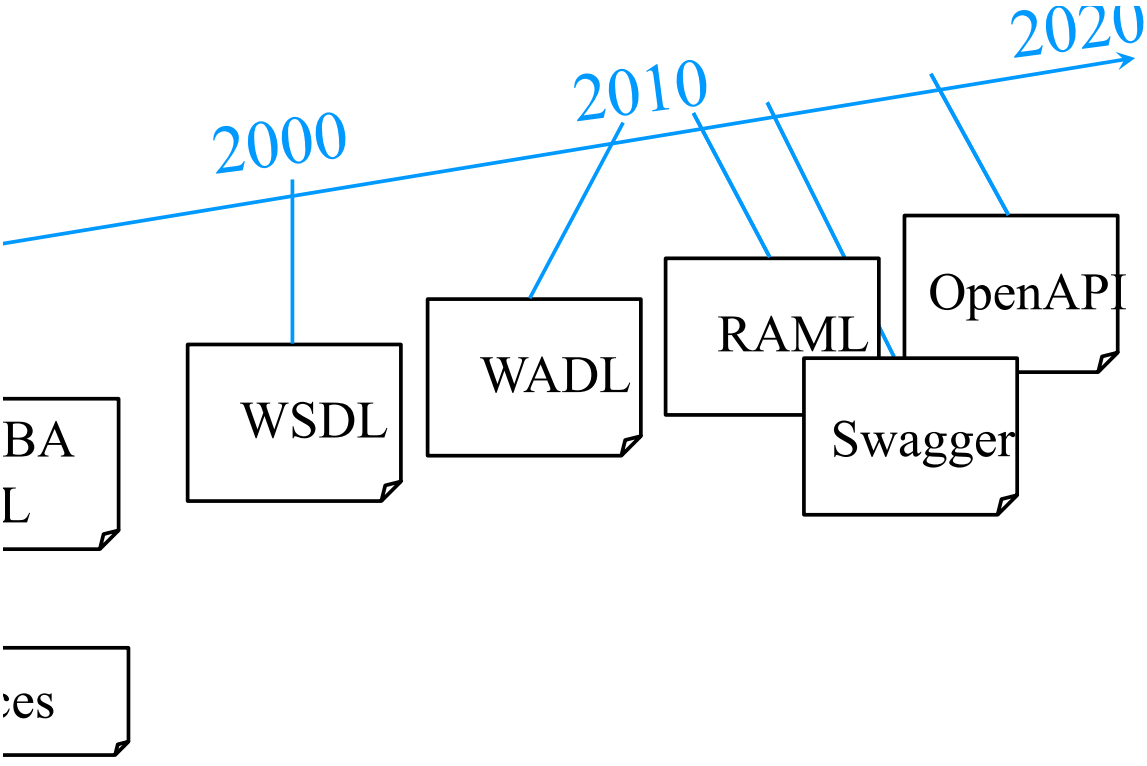
Sometimes determining whether your interface design is usable or not depends on who you expect to read and program against your interface. Sometimes it is a matter of style: what you're used to work with as opposed to what's a common convention while writing code in a given programming language.

Read interfaces, look for patterns, assess whether it's easy for you to understand how to use them, so that you can acquire a certain taste of what makes it an interface easier to use and to learn for developers that have never seen it before.

Interface Description Languages



Interface Description Languages



How do we describe such interfaces? Describing interfaces is so important that there

are specific languages designed just for that.

Sometimes these languages are a subset of an existing programming language, and sometimes they are completely separate. The only thing you can do with an IDL is to model the content of an interface: describe its data model, enumerate the operations, the events. and the properties.

If you want to implement the component behind the interface, you have to use a separate programming language, although it is possible to generate code from the IDL description (and viceversa).

Let's see from a historical point of view how IDL have evolved. The first IDL language was introduced to help with describing remote procedure calls. When you had a remote server separated from a client, then you had to have a mechanism to help the client know what the server was providing and be able to invoke it remotely, and so there was a specific language to describe all the remote procedures. You could already even generate both client and server code from it. This code would support the distribution of your system; when you move from procedures to objects, then you have also distributed objects environments like CORBA. Each of these middleware tools came with their own interface description languages. And also the Java language, which is an object oriented programming language, has a subset which is used to describe interfaces when you work with remote method invocation (RMI). This was very popular in the 90s.

Based on their names, you can see who is behind these technologies: what does the M in front of the interface description language stands for? Do you know which company implemented this technology?

You might have heard about it even if it's really ancient history: the M stands for Microsoft. Yes, good guess. So this was a Microsoft technology. Java was meant as an open alternative: an attempt to make "platform independent" software portable beyond Microsoft platforms. Then we had also this other group of middleware vendors which standardize around the CORBA OMG standards. While CORBA and Java did become interoperable at some point, there was never any interest from Microsoft to make their distributed object technology compatible with the one of the rest of the world.

So there was always this gap, this fragmentation in the market for software components, with interfaces that are either of these kind or that kind but there is no way that we can make them talk together.

Then Web services came out: the HTTP protocol and the XML representation syntax were used to solve the interoperability problem within distributed systems across the Internet. It took more than one decade and then finally here, you actually had widespread industry support for the WS-* standards, and you had both Microsoft implementations, and non Microsoft implementations, so that was the big step forward.

So we had the web service description language (WSDL) used to describe how to compose XML messages that could be exchanged between different remote APIs. A very interesting feature of this language, thanks to the fact that it was XML based, was that this language was independent of the programming language used to implement the service component on the server side and also the client program that would invoke it.

That was the time when XML was the most popular, so hyped that it took many years before people realized that XML was actually rather inefficient to use it as a serialization format to transfer data between different network hosts and mostly performance will be spent parsing complex XML documents, which tend to also have a low information density leading to larger message payloads.

Slowly people started to realize that it was possible to actually solve the same problem by using other – still text-based – formats, like JSON, or other binary representations like protocol buffers.

The languages for describing interfaces changed accordingly, from the web service de-

scription language (WSDL) to the web application description language (WADL), which was not so widely adopted.

While the underlying HTTP foundation never shifted, there were other attempts and iterations (RAML, Swagger) to describe HTTP-based or sometime also RESTful APIs leading to the current OpenAPI standard. We are now in the 2020s, but we still depend on HTTP port 80 being open by default through firewalls to transfer data and remotely access our software components delivered as a service.

Java/RMI

```
import java.rmi.*;
public interface Adder extends Remote
{
    public int sum(int x, int y) throws RemoteException;
}
```

Let's look at some examples.

Here is a description for a Java remote method invocation (RMI) interface. This is actually a piece of Java code. You know that this is an interface description because there is a specific language keyword: interface. You know that this is a special kind of interface that is remotely accessible because you extend this marker interface called Remote and you import the RMI package, which will give you access to the Remote and the Remote exception.

Why is it important to include this boilerplate code? While if you call such a method on a local object, you would expect that such call will never fail, the moment that you try to do this across the network, you have a lot of problems that can happen, and so you want this information to be visible in the code. Warning: During the call, there is network communication going on and things may go wrong, so you have this exception which may occur.

If you read the content of the method signature, this is a classical interface description for a typed language: You need to specify names and types for all operations and parameters.

C/RPC

```
interface adder {
  const long ARRAY_SIZE = 10;
  typedef long array_long[ARRAY_SIZE];
  long sum([in] long a, [in] long b);
  void array_sum([in] array_long a,
                 [in] array_long b,
                 [out] array_long c);
}
```

Parameter direction: `[in]` Input, `[out]` Output,
`[in, out]` Input and Output

Let's look at something more ancient, more exotic. This is the original RPC IDL which was used for describing calls to remote procedures, mostly implemented in C.

The syntax it is also similar to C, with a few extensions, starting from the keyword interface, found in prominent position.

Interfaces have names, so you can tell which server is offering which set of calls. Inside the interface we have type definitions. So that's about describing the data that we can work with, and then we also have operations, which look like function signatures, but there is no implementation like what you would find in a header file.

One function adds two long parameters. The second can process arrays; because of the intricacies of the C language where arrays are passed with pointers, when you transfer data across the network you cannot just send the pointer. This interface definition language was trying to abstract how the data will be copied around and where it will be stored.

In addition to names and types, you can use annotations for describing the direction of the transfer of the data between the remote function that implements the operation and the client. So in the first case this is a normal function. The parameters behave as expected. We pass the input and the result comes out from the return value of the function.

In the second case, since we work with arrays, the only way to get the output was to pass two array parameters as input and use the third parameter to receive the output array. This is kind of unusual or unexpected to call a function, but only the first 2 parameters need to be set while the third one is going to contain the result after the function returns.

So this a simple example of the origin of a whole family of interfaces description languages: we can already recognize the need for describing data types, and in this case we enumerate different operations. We have also this concept to describe in which direction we exchange information.

RAML

```
/adder:
    get:
        description: Add two values
        queryParameters:
           a:
              type: number
              required: true
              example: 40
           b:
              type: number
              required: true
              example: 2
        responses:
           200:
             body:
               application/json:
                example: |
                   {
                     "result": 42
                     "success": true,
                     "status": 200
                   }
```

Let's fast forward to this century. Here we can see how we use HTTP to access the same interface.

This is an example of a language not only used to give us machine processable information (for a compiler to read), but also some human readable description to express goal of interface. These interface description languages were born from the need to generate documentation for the interface: describing its features for developers attempting to first understand what it does and second how to use it. As opposed to just reading the Java-like, or C-like code, like with previous older attempts, here developers read colorful, interactive HTML documentation, which is generated from the model of the interface represented in RAML, Swagger or OpenAPI.

We can see in the example how URLs are used as an identifier for the different operations of the interface, so we can distinguish them because we have different links, different Web addresses pointing to each of them.

Since we communicate over HTTP underneath, we need to describe how to send a request with the GET method. Since we want to add 2 values, we need to pass these parameters, which have names and types. Here we also note that they are required. It's also possible to have optional parameters, for which you should provide sensible defaults.

Another useful technique to help developers understanding how to use the interface is to provide a concrete usage example: try to call it with these values and see what happens. This could also be considered as a simple test case, to make sure that given this input you get the expected output.

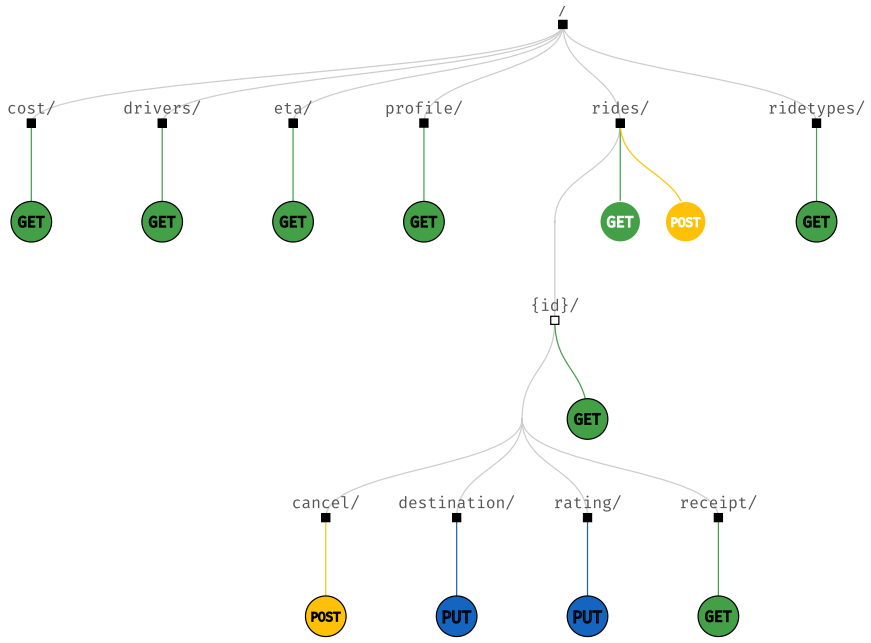
We also have information about the response, which will carry status code 200. And it will contain a certain body payload representing the result using JSON as a data format. And here is a concrete example of the object, which contains ther result, but can also be used to represent error codes in case something went wrong during the remote interaction.

So this is an example where the networking world comes into contact with the software interface concept. Network protocols are used to transfer information back and forth. We

reuse this ability to encode and transfer information so that we can process it with our code and not just use HTTP to download the pages of a website for people to read, but in this case we want to use HTTP to be able to compute a function remotely on the server and – for example – add 2 numbers. It is a bit overkill to compute an addition remotely on a remote server in the first place, no matter whether using HTTP or some other more efficient protocol.

But this is a general problem of distributed component technology, it has become so easy to split your architecture into multiple components and distribute them around the Internet. Sometimes we get carried away and decide that it is actually a good idea to make a remote call to add two numbers.

OpenAPI/Swagger



swagger.yaml

Let's look at a more complex and meaningful interface example. The OpenAPI code is available at the link, and you will find that it uses the latest fashion, the YAML syntax.

This is a more complex interface which we simplify and visualize on the slide. The tree visualizes the resource addresses, and their corresponding HTTP methods. If you want to get a ride from this particular provider, you can get the cost for the ride. Why do you need to use get? How much does it cost to go from here to there? These are geographic locations, modeled with planetary coordinates: latitude and longitude. Given these parameters, the result of the get method will return an estimate for the cost. The actual content of the response payload is defined in the data model for the interface and only referred to from the operation definition.

Other operations return the list of nearby drivers together with an estimation of – given where you are – how quickly somebody can come and pick you up.

If you're interested, you can scroll through the YAML code and you can get an idea of how to work with this particular API. This is just a summary as a visualization that shows you the structure of the paths that you can access, or the various features of the interface, which in this case are not a flat list of operations, but they're structured, nested along a tree.

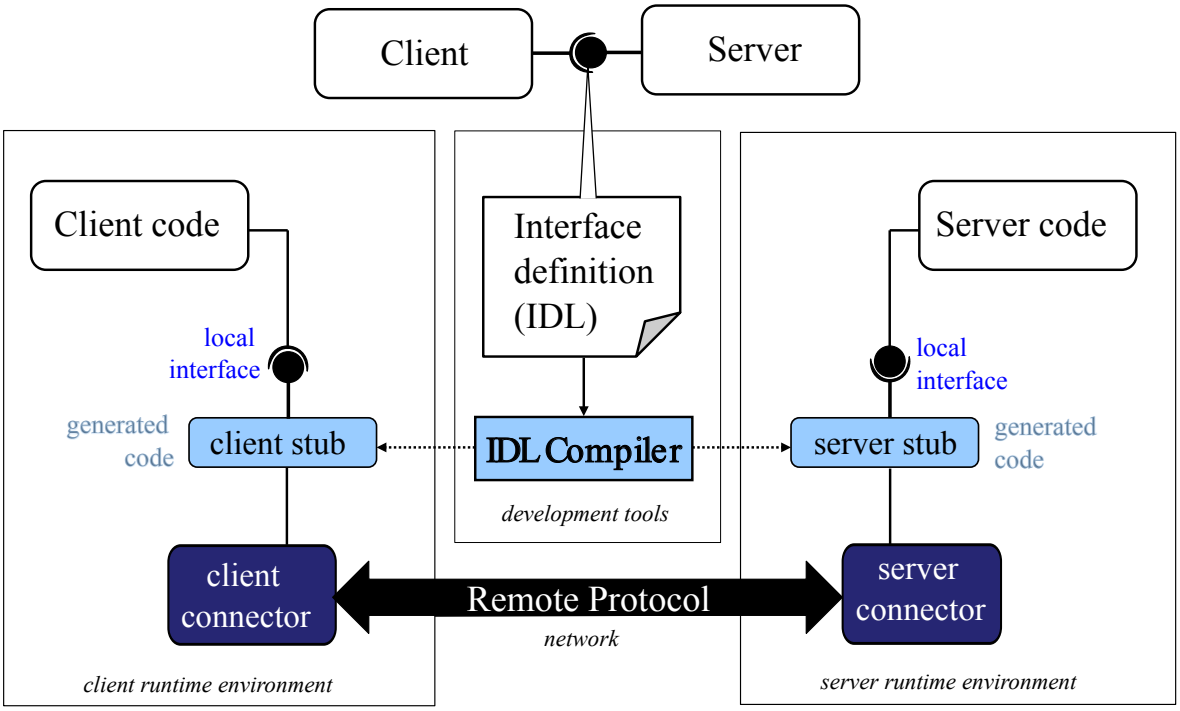
Regarding drivers, the estimation for how long it takes them to arrive, the information about our profile is all read-only information. The only method that we could apply was GET. This part of the interface give you access to read information, but you cannot change it. It makes sense, because based on where you are you want to know how long it takes for somebody to pick you up: this is not going to change the state of the system behind the interface.

However, if you book a ride, that's where it gets interesting, because you have both the possibility to access the list of the rides that you've done in the past, but there is also a POST method. This is something that allows you to place an order for a trip: given

this origin and destination locations, I want to book the trip. That's when you create and modify the state of the component, which is represented as a collection of rides; inside the rise you have their IDs. This gives you access to each ride individually, so that you can get detailed information, You can also get a receipt (that's also a get method - a read only operation). But we can also give a rating for the drivers. So this would be a modification that we do on the content. Like if we want to update the destination or cancel the ride altogether. For some reason they didn't use a DELETE, they use a POST method again.

Given the model of an interface specified using OpenAPI, it is possible to analyze it and summarize it using visualizations like this one, which can be very useful if there are hundreds of resources like in some of the largest interfaces that you can find if you search for more examples of OpenAPI descriptions.

Working With IDL



Once you have such detailed description of an interface, what can you do with it? On the one hand, you can generate documentation for people that need to understand how they can use your component. Having documentation, examples and getting started tutorials is key for them to learn how to use your component.

On the other hand, describing the interface using such a precise language makes it possible to actually generate code from such a specification with a compiler that can translate the interface definition in something that helps you to develop the implementation of the component on the server side. But it can also help you to write code which can call the interface on the client side.

The blue part is actually a component that is automatically generated by what you can call the IDL compiler or a model to code transformation. It takes the model of the interface as input and produces the code that will help you deal with the remote communication, code which abstracts away the fact that the two components are deployed across different containers so that the code you write in the White Boxes can pretend to call a local interface.

The actual distribution is managed by the middleware infrastructure so you don't have to solve the problems of: How to transfer arbitrary pieces of data from between two computers? How to process multiple incoming concurrent requests? How to guarantee that data is not lost in transit? When you deploy components in a distributed setting, this is all infrastructure that will do this for you.

As long as you can describe the interface precisely enough, these issues are taken care of. When your client code makes a local invocation, the generated code is going to take the input, transfer it across the network. The server will deserialize it. Pass it on to your implementation, and then when the server-side implementation has a response ready, the same will happen on the way back.

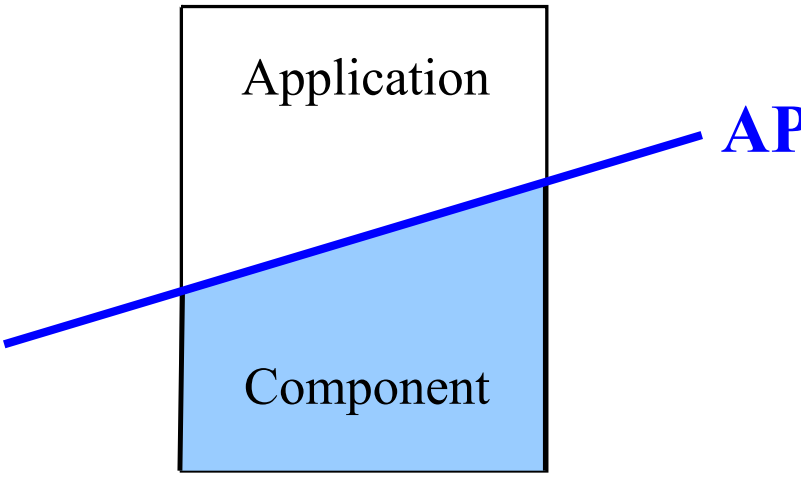
We have seen a very important application of the concept of modeling interfaces, which

is of great practical help as we deploy components in a distributed execution environment.

API Documentation Demo Videos

- How is the API documentation structured?
  1. Show an example of an existing API and go over its main elements.
  2. Which kind of data model or schema information is included with the interface description?
  3. Which kind of extra-functional requirements can be described?
  4. Is there some versioning metadata?
  5. Is there some API repository where such descriptions can be found?
- Given an API specification:
  1. How to generate a developer website?
  2. How to generate client code to invoke the API?
  3. How to generate tests?
  4. How to generate an implementation stub for the API?
  5. How to compare different versions of the API?

What is an API?



After introducing interfaces in general, now we focus on one special kind of interface which are so called application programming interfaces (APIs).

When you design your architecture, you have many components connected together: precisely the interface encapsulating and abstracting the components are connected. The interface provided by one component can satisfy what the required interface of the other component needs.

For one architecture there will be one particular kind of interface, which allows to connect the system with the rest of the world, across the overall boundary of the context view.

While every component has an interface, every architecture has a special interface called the API, which opens up the architecture and makes it possible to connect it with other systems that somebody else has developed at a different time and place.

Also, when you have a platform, when you have an operating system, or some programmable infrastructure software which can run different applications developed by someone else on top, the interface between the platform and the applications is also called the application programming interface (API).

What is an API? It's an interface, yes, but is an interface that has a very precise purpose which is to allow external developers to build applications on top of the interface.

You can see the API as a slice through your architecture which separates the general part from the specific part. The API separates the part that you have to implement it behind the API from the part that somebody else can implement for you.

If you are going to launch a Web-based service, you typically are responsible for implementing the server side. But if you choose to open up your service with an API into your system, then you can let others build the clients for you, while keeping full control of the interface you provide. So you can wait and see which clients become more successful, which ones have the best user interface, which ones gain more traction with a growing

user community. And then you can buy them. And through the API, of course you have one point into your architecture, which will make it possible to talk to you system, but you also make it possible for you to control who is talking to your system, check whether they are allowed to do so, and extract rent from your position.

Is it really API?

1 2 3

- You know you have designed a reusable API only after at least three applications have been built on top of it
- A component interface with only one known client does not qualify as a reusable API

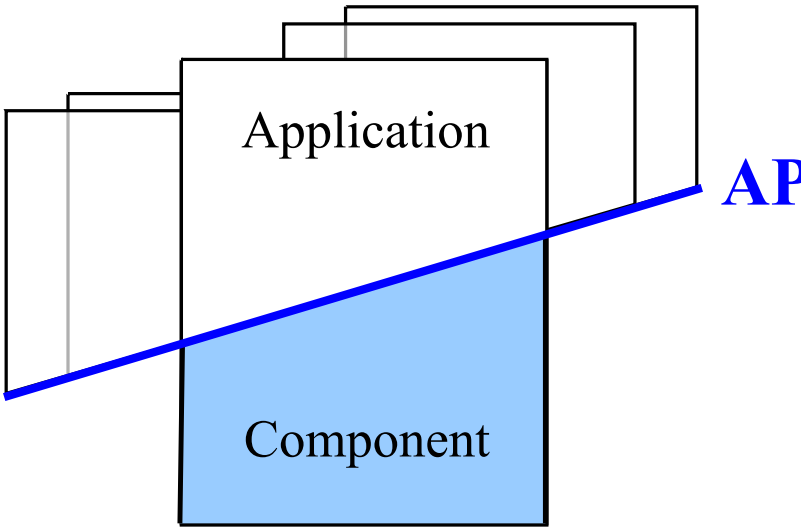
Does any interface of any component in your architecture share these properties? Probably not. So there is this rule, called the rule of threes, which is very simple to understand. It just says that you can claim to have designed a reusable API, if there are at least three applications which have been independently built on top of it.

You can always have a component which provides an interface that is used by one other component within your architecture. That's already enough to call it an interface because it abstracts the implementation of the component and makes it possible for the other component to access it.

But if there is only one known client, this doesn't make it a reusable API. If we want to have a claim that this is a reusable and you have to have at least three known uses, which represent applications built by someone else.

After all it's much easier to define an interface if you are the user and the implementer at the same time. It's much more difficult to do so while opening up your system so that others can use an interface that you provide and build their own applications using it.

Many Applications

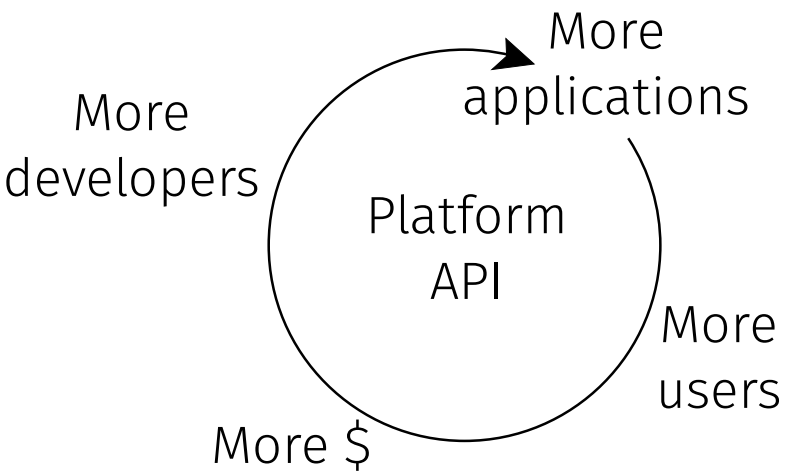


APIs thus are not found in all architectures, but only in those designed to be open and stable platforms supporting externally developed components and applications.

The API is key to make this possible, as it opens up the underlying platform components to the rest of the world, while making the developers that build application more productive, since they can reuse what the platform provides them with.

Because they don't have to solve the hard problems that you solve for them inside the component, they just use the API and can benefit from your efforts.

Developers, Developers, Developers



So we have many applications built on top of an API: but where do the application come from?

As Steve Ballmer once said: Developers, developers, developers, developers, developers. He was in a conference room crowded with developers, they are all developers for his API: their job is to build applications for it.

If you want to be successful with your platform and you have defined a ”good API“ for it, you will be interested to start this positive feedback loop.

The more applications, the more users will use them, the more developers will be interested to write more applications, for more users.

What is the incentive? Somewhere in this loop, the more people you have using the applications on top of certain technological platform, the more money you can make with it.

Developers will be able to sell applications to users and at some point along the way the platform will take a cut – like the 30% Apple tax.

Users will pay to install the platform, will pay to buy a license for the apps. Again, the more apps you have, the more user will come, the more money you will make, both as a platform provider and maintainer of the API, as well as a humble application developer, both enabled and supported but also limited by what the API can do.

Where to find APIs?

- Operating Systems
- Programming Language Runtimes
- Hardware Access
- User Interfaces
- Databases
- Web and Cloud Services

There are many examples of APIs and many examples of different kinds of APIs, which depend on the type of platform or underlying system they provide access for.

We will see APIs of many different categories. For example, we have local operating system APIs, or standard programming language libraries, as well as distributed and remote APIs to interact with databases or Web service providers.

Operating Systems

- Win16, Win32, Win64
- POSIX
- Cocoa (OS/X)
- android.os

Why is it important to standardize the API of an operating system? What property do you get for your applications if the operating system has a standard API?

You can write multi-platform applications which are portable both across different operating systems, but most important across different version of the same OS.

Consider the historical backwards compatibility of Windows. Within the 64bit API, somewhere you can still find a copy of the Win32 and within Win32 there is a corner where Win16 is still there. This is a possible way to provide backwards compatibility so that old applications still work in the newer version of the operating system.

Standard APIs also give consistency because different implementations will work in the same way. If this is not the case, every OS upgrade becomes risky. Your applications may stop working or need to be reinstalled or recompiled if the API of the OS changes and the old apps are no longer compatible.

Programming Languages

Standard libraries of any language runtime

- C `#include <stdlib.h>`
- Java Platform API (SE, EE)
- libstdc++
- Python Standard Library
- .NET Framework class library

Hardware Access

Abstractions to program any kind of hardware device

- Graphics (OpenGL, WebGL, OpenCL, CUDA)
- Network (NDIS)
- Printers (CUPS)
- Device Drivers (WDF, I/O Kit)

The goal of these hardware APIs is to give developers an abstraction so they can program applications for a certain hardware device category without worrying about the low level details.

It would be too expensive to write applications embedding knowledge and assumptions about every possible concrete hardware device.

APIs make it possible to decouple applications from devices. Device drivers will map the API primitives to the specific device implementation.

User Interfaces

Widgets, Gadgets, Controls

- Tcl/Tk, Qt, GTK+
- Java Swing, AWT, SWT
- Windows MFC, WPF
- JavaScript HTML5 APIs

Databases

Standardized database access

- JDBC
- ODBC
- PHP PDO

Why it's important also here to access database through standard APIs? To keep stateful components independent of the specific database implementation which will store their state.

As long as it's a SQL database, you can use these API to send the queries in, retrieve back their results and in general access the content of the database, no matter which one it is. The example shows that while relational databases have standardized their structured query language (SQL) since a long time, this was not enough to decouple them from the applications, until APIs such as JDBC or ODBC focusing on the mechanism used to perform the actual interaction with the database were introduced.

Without these standards APIs, every database that you select will be tightly coupled into your component into your system and it will become impossible to switch the database from one type of product to another, because every database will have their own specific interface. This is currently the case for noSQL databases.

Web Services

Remote access through standardized protocols:

- SOAP/WSDL Services
- REST/Hypermedia Services
- JSON-RPC/HTTP Services

Examples: Google, Amazon WS, Facebook Graph, Twitter Firehose, Salesforce

Web service APIs are used to access remote components across the Internet (or the Web).

They make it possible to access these cloud based services like Google, Twitter, Facebook, Flickr, Amazon. (And many, many others).

They all use standard protocols so that it is possible for them to interact with client applications without having to re-implement the client for every possible communication stack and for every different provider that you want to interact with.

It turns out that the HTTP protocol is the default choice, given it is easy to use programmatically and on top of it, people have been building, for example, the ability to perform remote procedure calls (RPC) using JSON payloads, or to query remote data source using GraphQL.

Since the good old XML days, which saw the rise of the WS-* standards, now they are no longer very popular but still work in case you need to describe and invoke ”legacy” Web services.

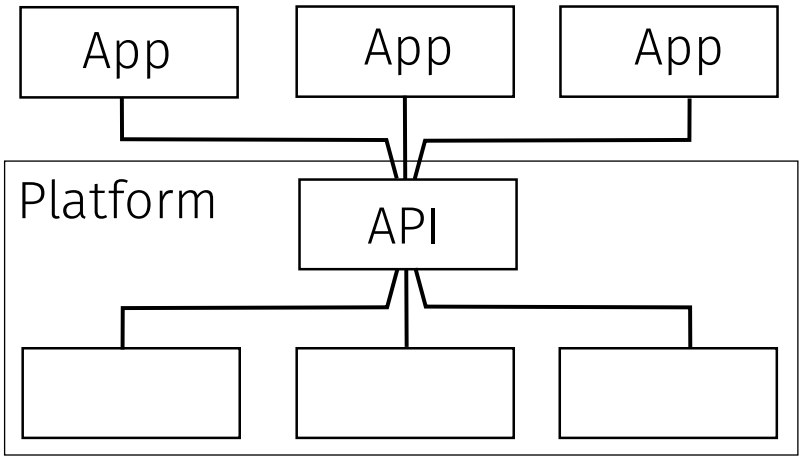
And yes, somewhere there are also those truly RESTful, hypermedia APIs, which most claim to be, but still very few understand how to correctly design or even what their purpose actually is.

I would like to spend the rest of this lecture to cover the topic of how do we design a good API. What kind of design principles can help us to make the right decisions? What are the options that we should consider when we need to design an API?

This is applicable as well to any kind of interface that you design, but since we said that the API is the integration point between your system and the rest of the world, it is also a critical part of the architecture that you have to get right.

The goal is to make it easy for as many clients as possible to interact with your system.

Where is the API?



Before we start, let me just show you a logical view with some anonymous components. Try to point out which of these is the one that plays the role of the API.

The API seems to be the one in the middle, the one through which all the other interaction have to go.

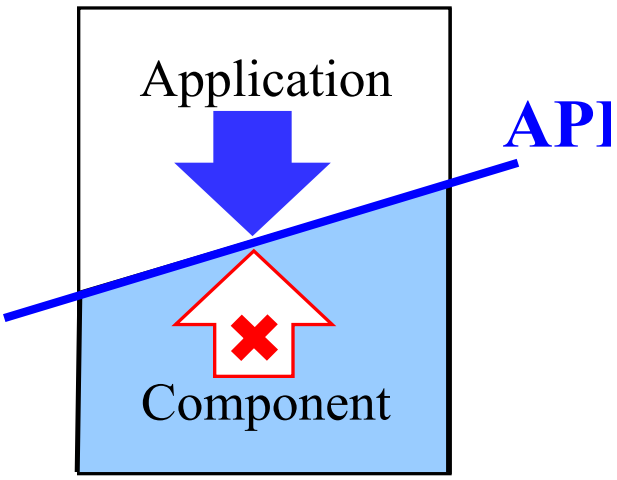
If we refine the design and for example, on top we have many components and at the bottom we have a single component. And if we open up the single component, the API is the one which exposes its internal implementation components towards the external ones.

Whenever you have this type of graph topology of your logical dependencies between components, the API should be easy to spot, even if it may not be yet explicitly called API.

This topology is also the natural result of applying the Strangler pattern, where you want to take a system in which there are a lot of dependencies, with components connected to many other ones and you gradually try to limit and control the connectivity by making all dependencies go through a dedicated separation point. For example, that's the place in which you can check, for example, whether the external applications are authorized to talk to your component.

This is also the point in which you abstract the complexity of the internal implementation to make your platform easy to access and easy to use for the applications, while keeping them isolated from its internal implementation components.

API Design: Where to start?



- The API should be designed from the perspective of the application developers using it
- Do not leak into the design of the API details from the existing component implementation

Here's your API: the main interface between your architecture and the rest of the world, the platform on top of which they built (or will hopefully build) all the applications.

So the other question I want to ask you is: if you are starting to design your API, and you are in this situation, cutting the system into two parts, where should you start from? Which side: you have your component, you decide what do you expose in the API. Or: you have your application that needs to access certain functionality and then you can use its required interface as the requirements to design your API.

Everyone agrees you should not start from the implementation. You do not want to leak implementation details and expose them to the interface.

Where you want to start from is the outside. You start from the application or in general you start from what the application would need for the API to provide.

You check your API design and evaluate it against the requirement of the applications which will need to use it. Once the AI design is sufficiently complete, we build applications with it. And then we can go and implement the underlying components, or map the API onto an existing implementation.

Outside in: never just take a component, click a button and make its private implementation accessible as its public interface.

The reason why this is not a good idea is that you want, after all, your interface to be an abstraction: you want the interface to hide the implementation of the component. You do not want the interface to be directly connected with the implementation.

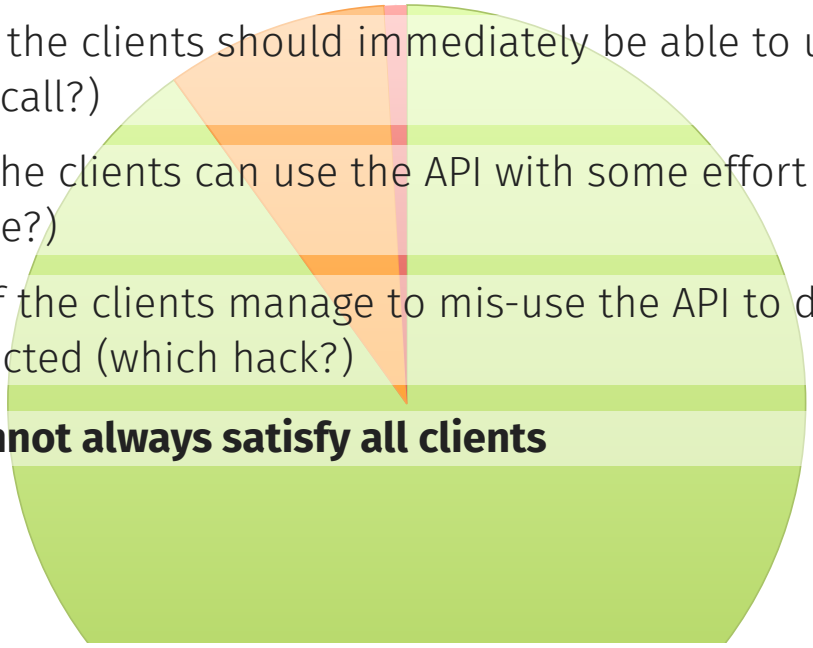
If you take the application developers perspective, your API will be easier for them to use.

Who cares about how it's implemented? You can even switch the implementation and if the API is good enough, it should not change and – as an added benefit – the application shouldn't change.

That's how you can also keep the backwards compatibility when you evolve your platform – for example, you change the operating system version, refactor its implementation – never touch the API and your apps will keep working unaffected.

Who to please?

90 9 0.9



- 90% of the clients should immediately be able to use the API (which call?)
- 9% of the clients can use the API with some effort (which example?)
- 0.9% of the clients manage to mis-use the API to do something unexpected (which hack?)

Note: **Cannot always satisfy all clients**

We need to start from the needs of the application, from the perspective of your customers. They want to build something on top of your platform. You start getting a lot of requirements. You have a lot of potential users that need to build many different kinds of applications. Can you really make everybody happy?

Well, the rule is that 90% of the clients should be able to use the API immediately. What does it mean? They find the right operation, the exact events, the precise property, which they can use to do what they need.

Maybe they need to look for it, but once they find, it is there and there they are fully satisfied.

A smaller group will need a little bit more effort to figure out how to use the API. They need to find a more complicated example or figure out how to combine different operations so they can achieve their goal.

And then an even smaller subset, will need to hack your API to satisfy their needs. They will to do something that from your perspective as a platform provider, is completely unexpected.

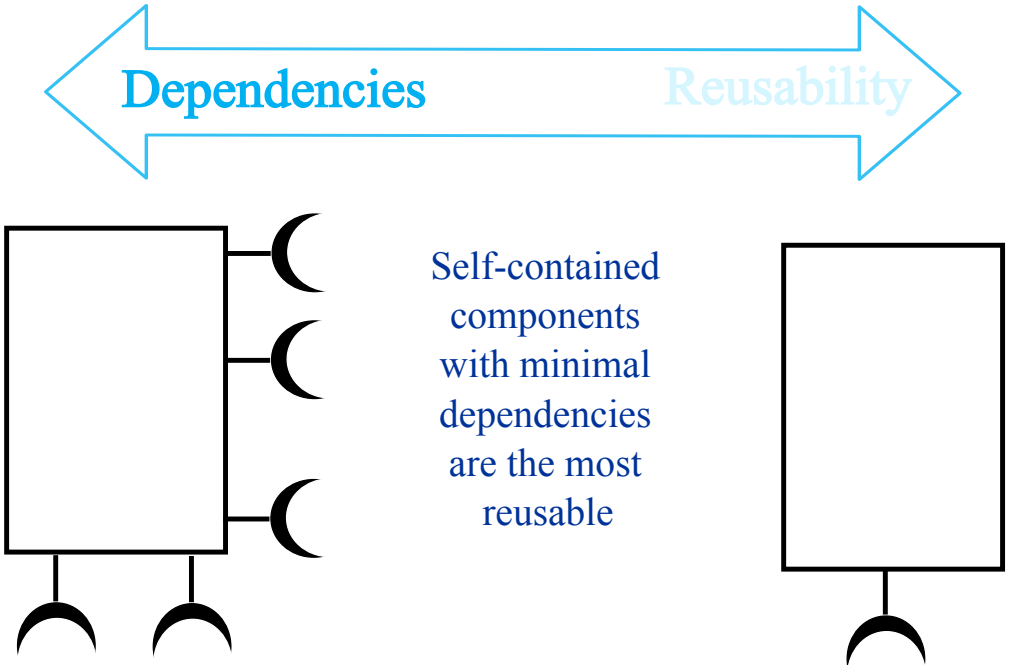
Overall the API shouldn't be too strict. Always leave some flexibility so that people can still solve their problems with it, even though you didn't anticipate those use case scenarios, but this is typically only true for a very small set of clients.

If they need to hack a solution on top of the API, maybe the API is not exactly designed for that so they're not probably so satisfied with what you offer, since they need to misuse the API to get results and will always feel that their hack may suddenly stop working.

The message is that if you have so many clients and everybody has different requirements, it would be impossible for you to satisfy everybody. So you need to prioritize. This is the same thing that we did when we had different use case scenarios in the do-

main model. We had many use case scenarios and we had to prioritize them, starting from the ones that are more common, so that by supporting them with your API, you can make the highest number of clients happy.

Reusable Interfaces



Let's look at the required interface. Which is the component that is most reusable? Do you pick the component that is on the right, or on the left?

The component on the left has a lot of Dependencies, while the other on the right doesn't have so many dependencies.

Increasing reusability is about minimizing those dependencies making the component as self contained as possible.

This is a also true if you build an API, if you have a platform, if you have a programming language, people should be able to build applications on top of it right away without having to install additional components and worry about satisfying too many extra dependencies.

When you are trying out a new platform, a new language or a new tool, what is the time it takes to go from zero to hello world? how many times do you need to fetch a missing package? or look up some cryptic error message to find a workaround? or does everything works smoothly the very first time you try it?

Usability vs. Reusability

- A general interface helps to reuse a component in many architectures
- A general interface is more complex and difficult to use in a single architecture

Back from the provided interface perspective, there is another important tradeoff to realize between usability and reusability depending on how general is your API design.

Since we need to satisfy the needs of many clients: how usable is your API for the specific client that needs to use it in a certain context?

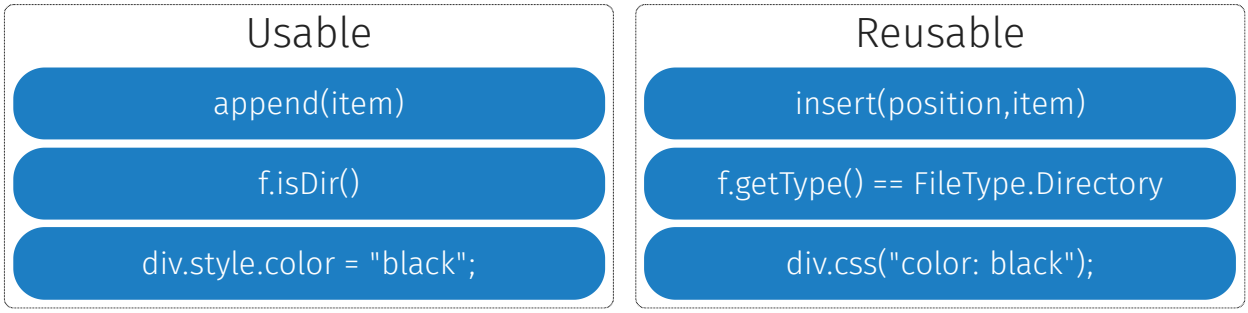
So to visualize this with a concrete example, you can look at these power sockets. When you travel around the world you typically have this problem of carrying adapters because different places will have different power sockets. Unless you go into one of these hotels or conference venues which offer you this type of plugs which can accommodate many different types of connectors. So this is a general ” power charging API“ in which you can plug in any kind of cable and it will give you the power that you need.

These other sockets are much more restricted: they only work within a certain type of plug. But if the plug exactly matches, for that specific use case, they will actually be easier to use. There is no doubt whether it will work or not or how to connect the two elements.

While a more general API supports many more clients, the problem is that for each specific client it could be a bit more difficult to use than a specific API designed and individually tailor made just for that one client.

Be careful how general your API becomes. The more general, the more difficult it will be to use for everybody. It’s a tradeoff: you need to find a sweet spot between how many clients you need to survive and as a consequence how general your API becomes without making life impossible for too many clients, or they will start looking for a better fitting alternative.

Easy to reuse?



Let's try to see how the trade off applies with these simple code examples. Which of the code examples is more usable for a specific scenario? Which of the code examples is more general?

The question is: how likely we can actually reuse the same interface without changes, when we find it a different use case?

Here we have two different ways to work with a certain data structure. We will append an item at the end, for example of a list. Or we can insert the items in any position, including of course at the end.

Everybody found the insertion the most general. Because after all, if you change the position, you can put the item where you want while appending is more specific and limited. However, if you want to put the item at the end of the array. Then you just call 'append' and that you don't have to worry about remembering: How do I identify the last position? Do I need to subtract 1 from the length or not? So 'append' is less reusable but more usable for the specific use case.

The second pair of snippets is about working with the file system. We have an object 'f'. We ask the object whether it is true of false that it is a directory. Or we can have a more general interface, where we ask the same object for its type and we compare the result with the expected value of the type, which in this case would be the directory.

For the third pair, we have the 'css' method that takes a string. The syntax of this string follows the rules of an entire programming language, which is – in this example – CSS. The actual CSS string is used to specify that the color of the element should be black. The alternative example does exactly the same, but uses a simpler, object-oriented approach, where given the page element 'div', you select its 'style' property, and use it to change the 'color' and directly set it to a certain value.

The boolean is as simple as it gets, either it is a directory, or not. This gives exactly the answer to our question, no more no less. However, if you have a different type of files. If

you have a symbolic link, a named pipe, or whatever other type of file your file system API may support, then you will need to come up with additional boolean methods for each kind. So you should consider the other, more general solution. Give me the type of the file and then you compare it with certain value and if you want to create a different type of files we just have to extend the enumeration of the types, but the method to retrieve the type does not change. The price developers using this general API have to pay consists of writing the condition to check whether the type matches the expected one by themselves, as opposed to embedding this condition within the API itself and simply returning the boolean result.

It's a trade off. And is it better to extend the set of file types that we support? Or is it better to add one more boolean 'is' method for every type we will ever want to support in the API?

This is a very specific example about the file system and working with directories, but you can see this in many different contexts. Do you want to have a boolean flag that checks the identity or the existance of a property? Or do you want to have a more general way too check what is the type out an extensible set of many possible values?

There is no absolute better or worse answer, you need to see from the client perspective. You need to consider how many possible values are there. Will the client developer remember where to find the set of possible values? Is this documented somewhere? Are you going to use an enumeration so that the compiler can catch wrong conditions? or will any integer do, so we have plenty of space for future extensions?

The one that is most controversial seems to be the one about the CSS. So in which way is this one more reusable than the other?

If we can pass a string (a string that of course has to be properly constructed, its syntax has to fit with a certain language, but from the point of view of the main language, this is just a string), then behind the API you will need to check that it has the valid syntax, that the property that you're trying to set is correct.

If you use the original language, it's more direct, just set properties following along a complex object tree structure.

However, that's the only thing you can do: You can set one property at a time. Here in the string version, if you want, you can pass a whole CSS file with lots and lots of property definitions, so this is definitely much more general and powerful, much more open for reuse but also even misuse.

Another place where we see this all the time is for example with database access, where you can send a SQL strings through the database driver. The database driver API happily takes any string and sends it on to the database. This is a different level of generality as opposed to working with something like where you have an object structure that you can access and then behind the scenes you make it persistent.

String APIs are so general that they can become security concerns: If you can sneak through arbitrary strings to be processed elsewhere, then your API is not able anymore to check whether these strings are allowed.

See for example SQL injection attacks, where after pretending to check user credentials you append an extra query to drop the content of the database. All the API does is take the string and pass it onto a separate component, which will execute it as long as it is syntactically correct.

Like when an entire schema declaration was bolted on the database connection string of a configuration file. The connection string is just a string, you will be surprised what desperate clients will write in it.

Performance vs. Reusability

- A general, reusable component may be less optimized and provide worse performance than a specific, non-reusable one
- For the same (reusable) interface, there can be multiple implementation components optimized for different hardware platforms/execution environments

Other tradeoffs that we have to consider when we work with reusable interfaces and APIs concern the opportunities for improving the performance of our components.

If you make a general component, like a platform independent component, for example, to make it reusable you may have to sacrifice its performance.

If you make a highly specific component, you can take advantage of its local environment, by requiring or depending on specific features which improve its performance, something that you may not be able to do if you need to keep it portable across multiple platforms.

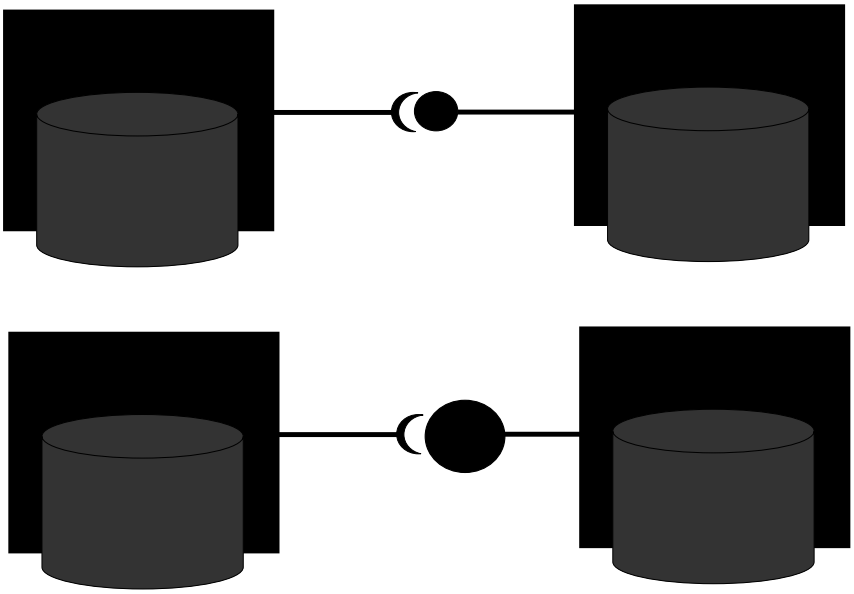
However, if we provide a reusable interface, then we can switch its implementation so the clients are not affected as we improve the performance.

Having an abstract interface, does not necessarily make it problematic to get good performance. It gives a constraint (the interface should not change), but behind it you can freely rewrite and optimize the implementation. For example, each implementation can use different processor instruction sets. You can have GPU-specific implementation, or an implementation which can take advantage of solid state disks.

There are after all Web servers running inside light bulbs, and some of the side channels exploiting processor speculation were written in JavaScript.

Sometimes if you worry too much about performance, you think you should not make your API too abstract because you will lose the ability for your clients to access important features of the underlying platform. There may be more layers of indirection and therefore performance will get worse. But the good thing about having an interface is that you have a clear constraint, so you can focus your effort on optimize the performance of the implementation behind the interface. And the clients will see a benefit without having to change because the interface stays the same.

Small Interfaces Principle



- If two components communicate, they exchange as little information as possible

As part of these API design discussion, I also want to introduce various principles that you should keep into account as you consider different alternatives for the design of your API.

The small interface principles recommends that every thing else being equal, pick the smallest interface.

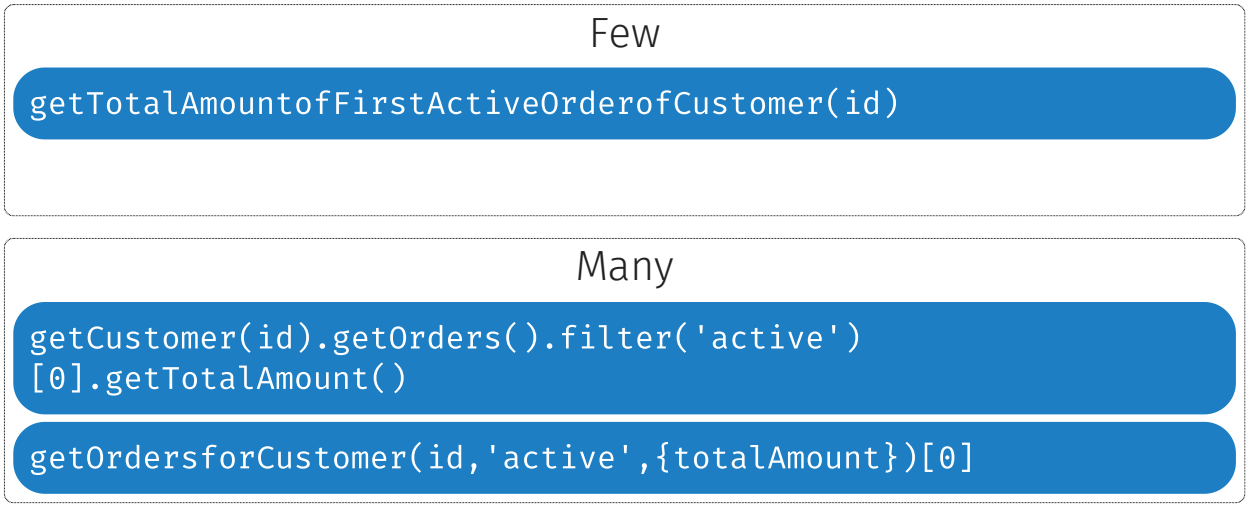
If two components communicate through one interface, they should exchange as little information as possible. So here we see two components connected. Here we have the provided interface. Here we have the required interface. They are connected together.

Here we have the same situation, they're connected, but this provided interface is bigger than it should be. Here we have a component that offers us a lot. It's interface is very, very expressive, very powerful, but we just use a small subset of that. The rest are a smorgasbord of features that are provided to us, but we don't really need them.

In this situation, the interface isn't as small as it should be.

A variation for this principle holds for programming language design as well: your language should be as expressive as necessary, but no more. Not all use case scenarios or application domains for a language require Turing completeness.

How many clients can these APIs satisfy?



To reflect about the small interfaces principle, I wanted to show you these examples. These are interfaces shown from the point of view of the client using them. I don't have room in the slide to give you a complete specification of the interface.

The use case is the following: there is a client that needs to retrieve the total amount of the first active order of certain customers.

While this is just one use case, as far as you know, and that's the only use case that you have regarding how you expect your clients to use the API.

When you design the API to make it possible for the client to get this information out of your system, you have these three options.

The three designs are not the same when it comes to satisfying potential use cases of other clients. Maybe there are other clients, who will also need to access the orders of a customer, but maybe they don't want to see the active orders they want to see the orders that are already paid. Maybe they're not interested in the 1st order, they want to read the 2nd order.

Some of these designs will make it possible to access the other information. Others are very narrowly focused on the needs of the original client.

In particular the 'getOrdersforCustomer' is meant to mimic something called graphQL: as part of your request to the API, you describe the shape of the object that should be returned.

This other design uses a single shot function, which has a very long name: its result gives exactly the total amounts of the first active order of the given customer. One perfect call doing really a lot of stuff for you, so that you get back exactly what you want. But that's probably the only thing that it does. If you need to get some other property of the order, there's no way to read them, this only gives you the total amount. If you want to get the 2nd order, sorry this returns only the first.

Now if we go to the other side, this is a more traditional design: we start from the

customer ID, we use it to retrieve the customer for that particular ID. Then this object will give us access to a collection of orders which we can filter. Because we need to just get the active ones. Then we take the first and finally we can retrieve the total amount.

With this solution we leave the door open for retrieving other information about the customers: we could get their address, we can fetch a collection of orders that can be filtered by many other criteria. And once we have an order object, we can get the total amount, but also we can get a list of the items that have been ordered. We can get a lot of different information about the order.

What is a drawback of using this solution for the API design? How many calls do you need to make to the API if you want to get to your specific result?

What's the minimum: one call. What's the smallest result: one value, the total. What's the maximum: four calls. We start from the customer's ID, then we get the orders. Then we do a filter (this could be a local operation), but still more work to do and then we get the order's total amount. So there's four steps. And every time you do one of these calls, there may be some remote interaction, some data to be downloaded, possibly encrypted, overall it can be rather inefficient. Think about the filtering: why should we retrieve the entire order collection, if we need to pop the first item only?

This also an important trade off when you design an API. Do you want your clients to access the whole data collection and then they are responsible for filtering it. Or, since the data collection is huge. And they just want to filter out a very small subset, like – for example – 1are active and the rest is all the history. So why should you have to download the whole history of the orders of this customer that has been with us for the past 20 years, when you just want to see the last active order. Maybe this is also relatively inefficient as a solution, and you want to perform the selection and projection of the data behind the API.

Another strategy would be to make the API less general, so in this case we want to get the orders for a customer in one step. This would take the customer ID as parameter and a filter condition for identifying which subset of the orders to retrieve. And also we want to have the total amount which is basically something that will project the result and limit its content. According to the API data model, an order can contain a lot of information, but we just want to read this part.

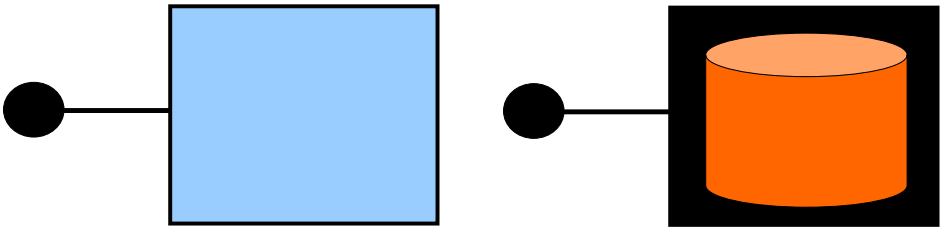
So here we have the best of both worlds, We have one call featuring server-side filtering both in terms of selection (the active orders) and projection (the total amount). Still, the API design is flexible, as we can change the order attributes as well as the selection criteria.

This filtering of the first element is something that we still do on the client side. We could also have a paginated result, where we start from the first result and incrementally retrieve the rest of the elements one by one or page by page. This can help to avoid downloading upfront lots of unnecessary data. That could be a possible design compromise if the collection is too large.

I hope this example was interesting to reflect about the different options that we have when we design an API, we can make it super specific and make one particular kind of customer very happy because they find exactly what they need. But maybe this is too specific, so no other client will be able to use the interface.

If we need to come up with these type of operations for every possible combination of queries that we want, we want to try to generalize the API and make it more complicated to use for a specific case because you need to know which parameters you have to pass, which are the valid filters and how to further process the result. But we can still keep the maximum number of round trip interactions and the amount of data to be transferred under control.

Uniform Access Principle



- Facilities managed by a component are accessible to its clients in the same way whether implemented by computation or by storage
- Helps to introduce caching or memoization without affecting clients

Bertrand Meyer

Another API design principle is about the abstraction of the interface with respect the implementation of the component. This is the uniform access principle: no matter whether functionality is implemented by computation or storage, it should be made accessible in the same way to its clients.

Why would you want to be able to do so? Well, the clients do not have to know if the result of their requests is computed on the fly or if it is actually stored somewhere, and you just retrieve the information that corresponds to their input, and you give it back.

If clients are unaware, then you have the opportunity – behind the interface – to do interesting time vs. space trade-offs. For example, let's make your component faster by caching, or doing memoization of the results, as opposed to wasting time, CPU cycles and melting the polar ice caps by recomputing the same results every time.

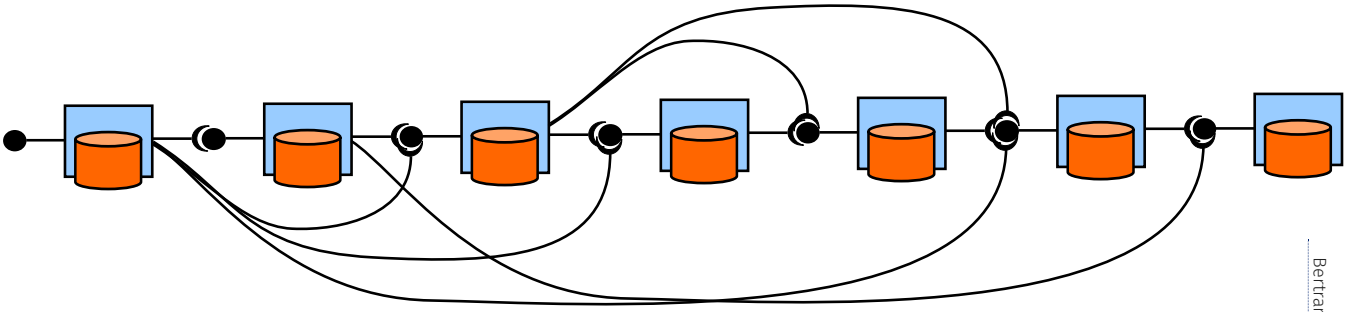
The result given to the client will not change, the way used to interact with the interface will be the same, although the performance may improve, so the client may be able to tell if the first time a request is sent the result is slow, and the next time the same result is much faster.

From the implementation point of view, you have the opportunity to decide whether, for example, some requests arrive so frequently that it makes sense to save CPU by storing the results and returning the same precomputed result to multiple clients. Is there enough storage to do that? Maybe you don't have enough storage so you have to recompute certain results which are not so frequently requested.

If the interface remains the same, you have the opportunity to optimize the implementation performance. One way to do so is by recycling previously computed results (if enough storage space is available).

If you violate this principle and only later you discover that it may have been a good idea to add a cache somewhere between your growing clients and your limited capacity server, then you will need to change and refactor your clients before the architecture can scale to absorb more traffic.

Few Interfaces Principle



- Every component communicates with as few others as possible

Bertrand Meyer

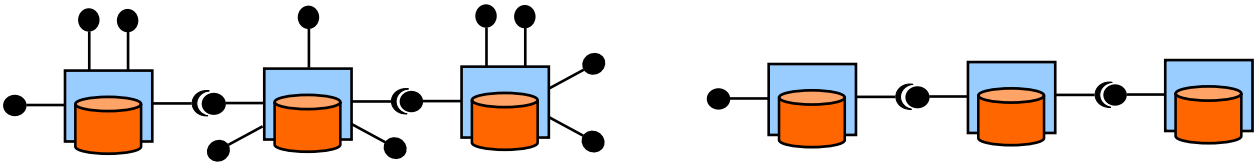
Another principle has to do with the amount of interconnections found between different interfaces.

While interface isolate the impact of changes applied to their underlying implementation, sometimes interfaces themselves need to change.

Imagine a nice layered architecture where every component only talks to its neighbors. So if we look at one component, this component is connected with the one in front of it, but also with the one behind it. And this component here does not see the effect of changes done to any component behind it. All of these components can be changed without affecting this component.

If you don't follow this principle, like in the figure, if you allow all components to freely communicate with whoever they want as much as they want, then you will not be able to control and limit the impact of changes. Every interface you evolve will potentially impact every other component, which may depend on it: every change will propagate and ripple across the entire architecture.

Clear Interfaces Principle



- Do not publish interfaces with useless or unused functionality
- Design interfaces that are needed and actually used by other components

Bertrand Meyer

The clear interface principle recommends to avoid publishing useless or unused functionality in interfaces.

This follows what we discussed before: if a client needs it, it has to be there. But try not to over-generalize your design to support hypothetical clients which may never be there, or publish features within interfaces "just in case". Maybe in the future somebody will need it. Now, however nobody does. And congratulations, your API just got more complicated with no immediate benefit.

It has to be needed right now by some actual clients: then you can publish it in the interface.

You can see in the figure some of these provided interfaces are used by another component. However, some interfaces are there, but there is a problem: there is no component for them.

Why should you spend the effort to design, document, publish, operate and maintain an API, if nobody is using it?

They just make the life of who is trying to connect to you more complicated: to find the point in which they can connect, the feature they need to use, they have to consider all of these other interfaces which nobody else is using.

While we can pay the price of supporting multiple actual clients and increase the size and complexity of an API as a consequence of its success, there is no reason to make the API more complex if this cost does not get offset against clients benefiting from the additional features, which may clutter the API design.

Other formulations of the clear interfaces principle include:

- If in doubt, leave it out.
- YAGNI: You ain't gonna need it.
- DRY: do not repeat yourself.

The first means that if you're not sure whether something should be published as part of an API. Don't worry, you can always add it later, but if you add it you have to be sure about it, because it will be difficult to remove it later.

Should your car dashboard look like an airplane cockpit? That would be so cool, however: how much extra training do you need to become a pilot as opposed to getting a drivers licence? How much time do you need to spend in a simulator to learn how to use this interface? So when you design your own interfaces for your own software, be

careful not to end up in this situation where the developers that need to learn how to use your API would have to go through three years of training to understand all the options on how to use all of your operations. Chances are that they are not going to need all of them. And if you are doubting that, whether you will need it or not, leave it out.

The DRY principle is about removing redundancy in your API, make sure that your interface stays as simple as possible, drop repetitions, only give one easy way to get the same thing done.

Let's create a new Window

```
HWND CreateWindowExA(
  DWORD     dwExStyle,
  LPCSTR    lpClassName,
  LPCSTR    lpWindowName,
  DWORD     dwStyle,
  int       X,
  int       Y,
  int       nWidth,
  int       nHeight,
  HWND      hWndParent,
  HMENU     hMenu,
  HINSTANCE hInstance,
  LPVOID    lpParam
);
```

Microsoft Win32

```
JFrame(String title, GraphicsConfiguration gc)
```

Java Swing

```
window.open(url, windowName, [windowFeatures]);
```

Web Platform API

```
QWindow(QScreen *targetScreen = nullptr)
```

QT

```
NSWindow init(contentRect: NSRect,
styleMask style: NSWindow.StyleMask,
backing backingStoreType: NSWindow.BackingStoreType,
defer flag: Bool)
```

Apple Appkit

```
public Form ();
```

Microsoft .NET Forms

I want to show you a historical perspective on API design. Let's look at this example from the user interface domain: we need to be able to create windows to display information on the screen.

Let's go back in time when people were actually writing this in C. You call the 'CreateWindowExA' operation: What do you get as a result? a handle to the window, like a pointer to the object representing the window that later you can of course manipulate. For example, you open the window and then you can close it.

To open a window, what information do you need to pass? What is the difference between the style and the extended style? Then we have the class name of the window. We have the name of the window itself. These are strings. A window is a visible object on your screen with a geometrical shape: so we have some coordinates. Where do we put it? how big it is?

Windows are also related to each other, so every application has many windows open so we should identify the main window. Windows have a menu, they belong to a given process. And there are also additional generic parameters that you may want to pass just in case. So this is for future extensibility, so we do not have to change all the existing applications in case we want to add parameters in the future.

It is not so uncommon to have so many parameters in this type of APIs. We can track over time how this number evolves. This particular API is also influenced by the choice of the programming language, and you can also see a very strong impact of a certain naming convention that was used at the time.

Let's look at the competition from Apple. This had been more recent as an API, but it does the same thing. You can recognize the style, but here we have a dedicated type that contains the style mask. This benefits from the higher level of abstraction of the programming language. With the previous example, in C, we need to work with bits, and

bitwise operators to compose these values. Here we just have a data type that is called style mask.

These four parameters X, Y, width and height have been compacted inside a rectangle, so that's also an attempt to raise the abstraction, to make it simpler. And everything else has disappeared: We just have these these two parameters about how the windows can be drawn and the option to defer the actual creation of the window on the screen until later.

Let's go multiplatform with Java. Definitely we have a much more abstract solution. We just pass the title for the window. This will be similar to the name. And there is an optional object which contains some options for configuring the window, which resembles the style that you use in the other two APIs.

Have ever opened a pop up browser window in JavaScript? This is also very simple. You just write 'window.open', pass the link to the page that you want to open. This will be used to control what's inside the window. And then you give it a name and then you can pass an open-ended set of other parameters, for example, to control the positioning or other features of the window.

This is probably the only case in which you just make one call and can obtain a fully functional, fully populated Window, since you pass a link to retrieve its entire content. In the other APIs, you have to create the object and then you have to write code to actually construct all the display widgets, all the content inside the window.

You can see from the names that prefixes or letters are use to brand the API identifiers and explicitly connect them to the corresponding platform.

This convention was not followed by this other graphical user interface toolkit, which is also the most abstract. They have come a long way since the original Windows API, here we just need to create the Form object, no need to pass any parameter, unless you need to override their default values.

This was just an example to show you how the same concept has been designed into APIs overtime. You can see many attempts to come up with the simplest and the most abstract way to solve this problem across different interfaces, influenced by the programming language, by previous attempts, by the conventions of the platform.

You can estimate how complicated or how easy it will be to write a program that uses each API, with different levels of abstraction, to create Windows and display graphical user interface objects on the screen.

Click on the links to get more technical details about each operation and the meaning of their parameters, and also compare how the way APIs are documented has changed over time.

Expressive? No: Stringly Typed

```
String do(String something);
```

Usage examples:

```
do("1 plus 1") // returns "result = 2"
do("1 divided by 0") // returns "error: division by 0"
do("nothing") // returns "ok"
```

Do you really want your clients to parse the result out of a String representation?

Use the programming language type system (e.g., enums) so that the compiler can detect incorrect API usage

Resort to strings only for inter-language compatibility or tunneling

What's the most flexible and expressive API design? Here is the simplest and most general API you can design: it takes a string as input and returns a string as output.

If you want to use it, you have to, of course, pass the correct strings to it.

How expressive is this type of interface? It is rather difficult to answer if you just look at this piece of information. To know exactly how to use it we need to describe the input language and the output language and how they relate. The compiler will not help because as long as you exchange strings, there is no data representation problem. A string can represent any data structure, as long as it can be converted into a string.

These interfaces have been called "stringly typed", in opposition to strongly typed, which are interfaces which feature parameters and the result having a very well defined type. It is not that strings are not well defined, they are strings after all, but they can be super flexible to tunnel any serializable data type.

Strings can represent natural language text, numbers, data items, base64 binary data, markdown, error codes, exceptions, complex data structures serialized using JSON or YAML, or even XML.

But when you look at the interface definition, you don't know exactly what the interface expects as input or returns as output.

A chatbot could use this interface, this could be the API for a smart home assistant, which would transform speech into text and feed it to this interface. What can you tell to your assistant? Anything goes.

Strings can also contain executable code, for example SQL statements that you feed into a database, or JavaScript code evaluated by your trusted Web browser.

If you think about processing the results, do you really want your clients to use such an interface and then have to parse the actual result out of a string representation?

All you can do when you program your client against this interface, is to assign the result and store into a variable of type string.

If you need to process the result beyond printing it out, you need to feed the string to a parser, which can extract its content, based on some assumptions you make about the language defining the syntax of your string.

That's why type systems in programming languages have been invented. You don't have to do the parsing yourself, but the compiler helps you to structure the data and give it a representation which can be manipulated from your code.

If you have a string in your interface, does it have any impact on its compatibility? Strings are compatible with strings, so if all interfaces exchange strings, you solved all compatibility problems in your architecture, and everything can be connected with everything else.

What if you have a more constrained type system defining your interface data model, then you can take advantage of type checking for detecting whether this interface is misused or used correctly, whether it is connected with other interfaces whose types match or don't match. This check you can perform statically, based on descriptions, no need to actually call the APIs and observe whether they accept or reject your strings.

It's not possible to detect all possible incompatibility situation because sometimes even if the types actually match, there are deeper semantic issues. But they already help you to catch a quite a few cases in which you make a mistake.

Another important case in which strings are useful is if you want your API to be programming language independent. For example, Web service API exchange messages which are basically a string representation (e.g. in XML, or JSON). Somewhere you need to convert your data into strings and back, but this makes it possible to exchange data between different programming languages.

To summarize: How general can we go with the API design? One of the most general solutions, concerning data representation, is the one in which everything is tunneled through strings. Advantages are that you can make this interoperable across different languages. The big disadvantage is that until you receive the actual string, you cannot tell whether it is compatible or not. There is no static checking that you can do about compatibility or whether the right parameter is in the right slot because you don't know anything about the types that describe the data model of the interface.

Consistent?

```
char * strcpy (char * dest, char * src);
```

The strcpy() function copies the string pointed to by src, including the terminating null byte ('\0'), to the buffer pointed to by dest. The strings may not overlap, and the destination string dest must be large enough to receive the copy. Beware of buffer overruns!

```
void * memcpy (void * s1, const void * s2, size_t n);
```

The memcpy() function copies n bytes from memory area s2 to s1. It returns s1. If copying takes place between objects that overlap, the behavior is undefined.

```
void   bcopy  (void * src, void * dst, int n);
```

The bcopy() function copies n bytes from src to dest. The result is correct, even when both areas overlap.

Here are a few more examples to show you that when you design your API, it is also important to be consistent.

These three functions, they do exactly the same. They are used to copy data from one location of memory or another. In some languages you are doing manual memory management and there are some cases in which you have a problem of copying data.

They do not belong to the same APIs; can you spot the inconsistency? Look at the way parameters are positioned. Or at the parameter names.

For example, you need to copy a string into another one. You have to use this type of functions. As we observe these signatures we can see that the types are different: void pointers or character pointers, which make sense for copying character strings. We pass pointers to the source string and to the destination. If we look at the last one, the order of the parameters is actually the opposite.

One function asumes we can start from the source location and continue copying until we reach the end because the strings are terminated by the null (zero) character. Another function instead takes the number of characters to copy, so that's a slightly different convention. The length of the string is embedded in the string encoding. The length is given as an explicit parameter.

One function just takes two pointers: s1, s2. So based on reading this and your knowledge of C, do you know which one is the source and which one is the destination? Or maybe you can look up the natural language documentation for the function and that can help you. s2 is a constant pointer. This subtle detail tells us something important. That this is a read only pointer, because we cannot modify it, which is consistent with the parameter order of the first example.

Other differences concern not only the signatures, but the way that the implementations deal with overlapping memory buffers.

But what can be the impact of such inconsistencies? What could go wrong if one function assumes the source follows the destination, and the other function uses the opposite parameter order?

Have you ever done something that is called copy'n'paste programming? You write your code using the first function. Then you do copy paste. In the cloned code, you switch to the third function.

This happens to me quite often, I copy and paste, I change the name of the function, assuming that the parameters are fine, they do not have to be adapted.

If the parameter order needs to change and we forget, what could possibly go wrong? We switch the pointers for the source and the destination. Can the compiler help us The code that used to work, now it is actually copying the destination over the source, which can definitely lead to unexpected behavior, as it corrupts the memory heap of our program.

Consistency is important. Following conventions is important. Don't surprise your API developers by changing the order of the parameters. If you do, warn them, and use explicit parameter names: source and destination are much more clear than s1, s2.

Primitive Operations

Which is the primitive operation?

```
playSong(mp3)
playSongs(mp3[])
```

Primitive operations cannot be implemented by combining any other operations

Derived operations can be implemented as a combination of primitives and do not always have to be included in the interface

To help you choose what we need to include in an interface to keep the interface as clear and as simple as possible, we can use the notion of primitive operations.

Primitive (or atomic) operations cannot be implemented by combining any other operation you find in an API. Derived operations can be implemented by composing other existing API operations.

If you remove a primitive operation, there is no longer a way for the client to perform it. The operation is gone and the client has no alternative, no work around is possible.

Removing a derived operation, even if you remove from the API, will not break the client, as it is still possible for the client to access the same functionality by composing by itself the primitive operations corresponding to the derived one which was just removed.

The question here is between the play song and play songs (one takes a single song and the other one takes a list): Which of the two is primitive?

It should be possible to implement the derived operation based on the primitive one.

Play song: primitive. Play songs: derived, because it can be implemented using play song.

How would you do it? Just write a little loop: for every element, play song.

If the API can play one song for the client, it's possible to use it to play multiple songs.

What is your assumption when you write that loop? How can you claim that they are equivalent? What if the play songs will play the songs of the playlist one after the other?

So you can write using a loop, if you assume that play song will start playing the song, wait until the song is finished, and then return so you can call it again with the next song.

So we assume "play song" is actually a blocking call, which will play the whole song until the end, and then return. We assume in other words, that it is not just a call to start playing the song.

If this is true, then this is primitive and this is one just a derived operation, which we can drop from the API or choose to include if we want to provide support for playlsits.

If the assumption is false, then it is not possible to use play song to implement play songs, and both are considered primitives, both should stay in the API.

Simplify the API


```
playSong(mp3,from,until)
loadSong(mp3)
seek(t)
play()
stop()
pause()
resume()
togglePlayPause()
isPlaying()
getDuration()
getPosition()
setPosition(t)
onSongLoaded
onPositionChange
onSongFinish
```

Here is a more complicated example where it is possible to find different subsets of primitive operations.

To detect derived features, try to see if you can compose other features and obtain the equivalent behavior.

If – like the case of 'playSong' – the resulting composition spans many primitive operations and events, and if it turns out to be a popular operation, it will be definitely useful for clients to find it directly in the API and not having to figure out how to implement it themselves combining the basic primitives.

If we look at 'setPosition' and 'seek', these two look redundant, one would be enough, one can be implemented using the other (and viceversa). In this case which one do you keep? Consistency with 'getPosition' would recommend to keep 'setPosition', but 'seek' is also a commonly used term in the media player domain model.

Sometimes events notifications of state changes can be redundant regarding properties. Is it enough to be able to read the current state ('getPosition') or to get notified when it changes ('onPositionChanged'), or do we need both? This would give the most efficient usage of the API: read the state initially and then keep track of it as soon as it changes via the event? (no need to keep refreshing the position property).

Design Advice

- Keep it simple
  - Do One Thing and do it well
  - Do not surprise clients
- Keep it as small as possible but not smaller
  - When in doubt leave it out
  - You can always add more later
- Maximize information hiding
  - API First
  - Avoid leakage: implementation should not impact interface

Joshua Bloch

Design Advice

- Names Matter
  - Avoid cryptic acronyms
  - Use names consistently
- Internally Consistent
  - Naming Conventions
  - Argument Ordering
  - Return values
  - Error Handling
- Externally Consistent
  - Imitate similar APIs
  - Follow the conventions of the underlying platform

Joshua Bloch

Design Advice

- Document Everything
  - Classes, Methods, Parameters
  - Include Correct Usage Examples
  - Quality of Documentation critical for success
- Make it easy to learn and easy to use
  - without having to read too much documentation
  - by copying examples
- Make it hard to misuse

Joshua Bloch

We close with a summary of what we know about API design.

Simplify. Maximize information hiding. Outside-in. Abstract.

Naming conventions are fundamental; consistency of names and order of parameters is important. Also keep the API consistent with its environment and programming language conventions.

Everything has to be documented. Everything else being equal, if you have a high quality documentation, this makes developers more inclined to learn how to use your API and eventually adopt it.

Documentation should include examples, not just describe and explain features, but examples make it easy for developers to reuse them by copy'n'paste. However, developers should be able to be productive without having to read too much documentation: if you need to read 500 pages before you can write a Hello World, there is some problem.

With interfaces, we learned how to design reusable components. In the next lecture, we are going to connect them together.

References

- Olaf Zimmermann, Mirko Stocker, Daniel Lübke, Uwe Zdun, Cesare Pautasso, **Patterns for API Design: Simplifying Integration with Loosely Coupled Message Exchanges**, Pearson Education, 2023
- Joshua Bloch, How to Design a Good API and Why it Matters, Google Tech Talk **Slides Video**
- Michi Henning, **API: Design Matters**, ACM Queue, Vol 5, No 4, May/June 2007
- Will Tracz, Confessions of a Used Program Salesman, Addison-Wesley, 1995
- Jaroslav Tulach, Practical API Design: Confessions of a Java Framework Architect, APress, 2008, ISBN 1-4302-0973-9
- Jasmin Blanchette, **The Little Manual of API Design**, Trolltech, 2008

Software Architecture

Composability and Connectors

7

Contents

- Software Connectors
- Components vs. Connectors
- Connector Roles and Qualities, Cardinality, Binding Times
- Connectors and Distribution
- Connector Examples: RPC, File Transfer, Shared Database, Message Bus, Stream, Linkage, Shared Memory, Disruptor, Tuple Space, Web, Blockchain